Parametric Multimodal User Memory: Storing What Captions Cannot Carry

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

A personalized agent needs a *user memory*: a persistent model of who its user is. Today it is almost always *text* — transcripts and captions retrieved by similarity. This serves the *captionable* half of a person ("my cat is named Bibi"), but discards the *perceptual* half no caption can hold: how a voice sounds, how a face reads across age and lighting, how tired someone sounds. We measure this loss across five modalities: a strong caption-based re-identifier recovers as little as 0.11 of a dedicated encoder's recall, collapsing toward chance on non-nameable signals.

We instead **ground** perceptual memory in the model, decomposing recall into two subproblems: a vision-language model grounds the referent in context (*what* and *where*), and a dedicated encoder extracts an identity *key* (*who*), stored as one inline token read by attention at generation with no external round-trip. Neither suffices alone — the VLM identifies cross-age faces at only 0.54 recall where a face encoder reaches 0.81, and an ungrounded encoder recognizes a two-person-scene referent at 0.05 — yet together they reach correct-region oracle (0.96), generalizing to multi-speaker audio and video. The recognition core is *training-free*: it reproduces the encoder's recall on any frozen model at $\mathcal{O}(1)$ registration cost. On **PerceptMem** (12 domains, 1,080 tasks) perceptual identity is capacity-limited (recall $\approx \min(1, k/M)$ of the encoder's ceiling), and exact facts are limited by binding each code to its name, which a latent learns slowly and unreliably even with one slot per item: identity belongs in a parametric bank, facts in a text store. The two memories compose cleanly: an agent with both can remember not only what its user said, but also what they are like.

Code: https://github.com/19PINE-AI/multimodal-user-memory
Website: https://01.me/research/multimodal-user-memory

1 Introduction

When an AI agent "remembers" something about the person it serves, what does it actually keep? In almost every system today, the answer is *text*. The dominant conversational memory architectures—Mem0 [Chhikara et al., 2025], MemoryLLM [Wang et al., 2024], MemGPT / Letta [Packer et al., 2023, Letta Team, 2024], M3-Agent [Long et al., 2025], and related work surveyed in LongMemEval [Wu et al., 2025]—convert what the agent sees or hears into transcripts (via ASR [Radford et al., 2023]) and captions (via models such as BLIP-2 [Li et al., 2023] or LLaVA [Liu et al., 2023]). These fragments are stored in a sentence-encoder vector index [Reimers and Gurevych, 2019] and retrieved later by similarity. Similarly, personalized

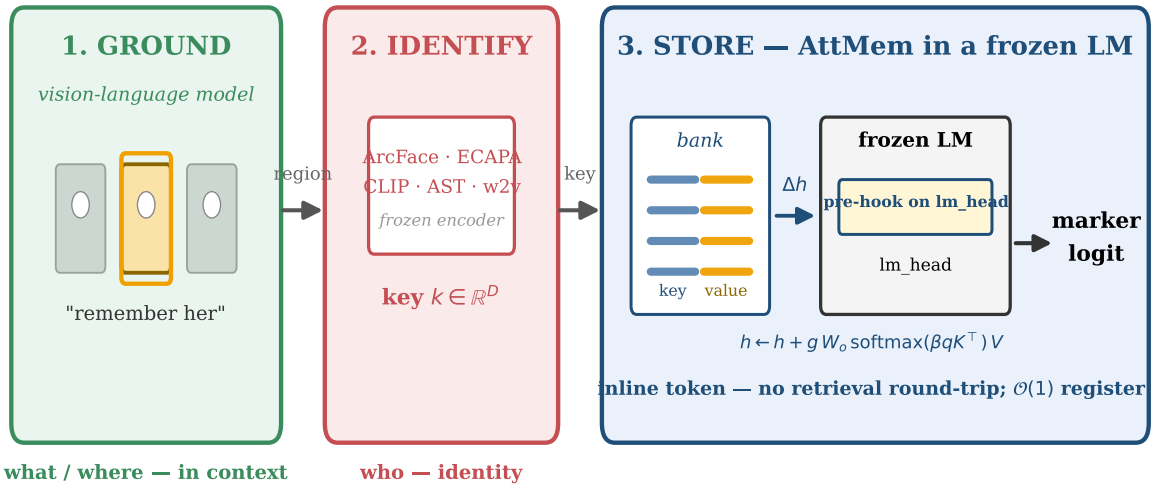


Figure 1. A vision-language model grounds the referent the user means in a cluttered scene (*what*/*where*); a purpose-built encoder turns the grounded region into an identity key (*who*); and AttMem stores the key on a frozen language model, reading it back as a single inline token. Neither component suffices alone — the VLM is a weak identity encoder, and a bare encoder cannot pick the referent out of a scene; grounded, they recover the correct-region oracle (Figure 2). The mechanism is detailed in Figure 4.

vision-language systems (MyVLM [Alaluf et al., 2024], Yo'LLaVA [Nguyen et al., 2024], MC-LLaVA [An et al., 2024], Online-PVLM [Bai et al., 2025a], RAP [Hao et al., 2025]) also rely on a textual backbone: a *named* concept (e.g., "Bibi") serves as the handle, while the underlying perception remains an afterthought.

This strategy works for the *captionable* part of a person — information like "my cat is named Bibi," "I am vegetarian" pass through text intact. However, it is not enough for the rest: how a voice sounds or how a face reads across age and lighting cannot be captured in text alone. The description "a brown-haired man" cannot tell two brown-haired men apart. Furthermore, a re-identification note from a strong captioner recovers a perceptual identity at as little as 0.11 of a purpose-built encoder's recall (Section 5)—the loss is a property of the channel, not the wording. This *perceptual* half is disregarded in current systems: it is flattened into a caption and lost, or is handed to a recognizer that returns a bare label like "speaker 47" which the model never sees. The missing piece is a memory that stores a perception *as a perception*.

Our proposal: grounded, parametric, multimodal user memory. We keep each perception in its native modality and decompose remembering into three steps (Figure 1): a vision-language model *grounds* the user's intended referent in context (*what* and *where*). **Grounding is the act of isolating the referent before encoding it.** A dedicated encoder (ArcFace [Deng et al., 2019], ECAPA-TDNN [Desplanques et al., 2020], CLIP [Radford et al., 2021]) turns the grounded region into an identity *key* (*who*); and AttMem, a small per-modality memory bank on a frozen language model, *stores* the key and reads it back by attention at the output head as a single marker token. Recall is then a token the model conditions on — produced inside generation with no caption and no retrieval round-trip, registered in one $\mathcal{O}(1)$ tensor append with no per-user training. AttMem borrows the attention-over-a-bank shape of kNN-LM [Khandelwal et al., 2020] and Memorizing Transformers [Wu et al., 2022], aimed at a new target: a user's persistent perceptual identity.

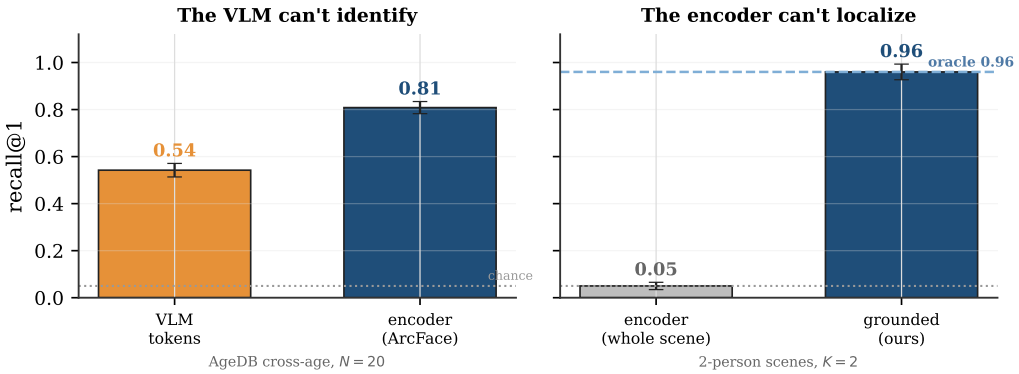


Figure 2. Neither component works alone; grounding covers both blind spots. Two targeted experiments isolating the failures of each component. *Left (identity):* on AgeDB cross-age faces (N=20), the VLM's vision tokens are a weak identity encoder (recall 0.54) while a dedicated face encoder achieves 0.81 recall. *Right (localization):* in two-person scenes (K=2), the performance of a bare encoder embedding the whole scene is close to random (0.05), while grounding the referent first achieves near-perfect performance (0.96). The division of labour is essential: the VLM excels at localization but struggles with identity invariance, while the specialist encoder is highly invariant but cannot locate the target. Detailed ablations appear in Figure 7.

Why grounding is necessary. A perceptual memory cannot simply encode whatever the agent observes, because a raw perception mixes the identity worth remembering with the context around it. The same friend photographed at a coffee shop and while surfing have almost no pixels in common, so a whole-image embedding would place them far apart. However, we recognise the person to be the same at a glance by *focusing on the face* and discarding the background. The same principle applies to audio: a voice on a phone call and one in a podcast differ substantially in terms of words and acoustics, yet a listener reliably identifies the speaker. This creates a division of labour (Figure 2): a vision-language model identifies *which* region to attend to but is a weak identity encoder, while a specialist encoder produces condition-invariant representations but cannot locate the referent in a cluttered scene. Grounding lets each component cover the other's blind spot, achieving recognition that neither can reach alone.

Figure 3 contrasts the two methods on one "remember her" task.

Contributions.

- **Perceptual user memory must be grounded, not captioned.** We argue and empirically demonstrate that text is fundamentally insufficient for perceptual user content. Effective recall instead requires a decomposition into VLM grounding, specialist encoding, and in-model storage (Sections 1–3).
- **A grounded memory that recovers oracle recognition in context.** By combining a VLM for referent grounding, a specialist encoder for identity extraction, and an inline attention token for storage, our method achieves near-oracle accuracy in cluttered scenes—where whole-scene encoding is near chance. This holds across vision, audio, and video (Section 5).
- **A training-free, $\mathcal{O}(1)$ in-model recognition core.** Our memory is read via attention at the output head with *zero* gradient updates. It reproduces the specialist encoder's performance across ten frozen model families, with constant-time insertion and recall. This is far cheaper

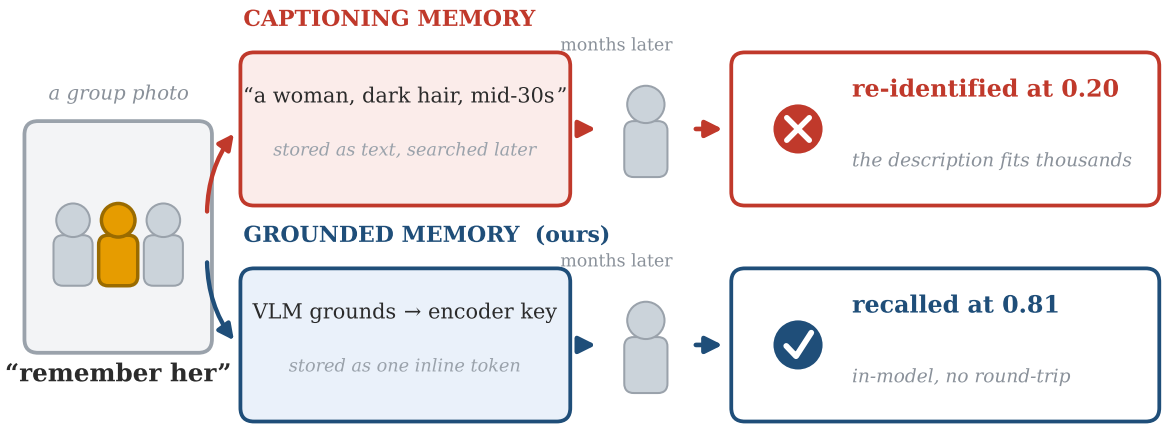


Figure 3. A concrete example. Given the instruction “Remember her” in a group photo, a captioning memory stores a generic description that matches thousands of individuals, leading to near-random re-identification performance. Our grounded memory instead stores a single inline token that supports accurate recall of the correct person across different conditions. The numbers shown are paired re-identification accuracies (Section 5).

than keeping raw perceptions in context (Sections 3–5).

- **PerceptMem benchmark and a capacity-law router.** We introduce PerceptMem, a benchmark spanning 12 perceptual domains and 1,080 tasks across five modalities. The benchmark includes text-only and random chance baselines. In addition, we measure what a latent memory of k soft tokens can hold. Perceptual identity follows $\min(1, k/M)$; exact facts are limited by *binding* each code to its name, which a shared latent learns slowly and seed-dependently and which per-item slots help but do not guarantee. Together they motivate routing perceptual identity to the parametric bank and exact facts to a text store (Sections 4, 8, Appendix C).

2 Decomposing perceptual recall: grounding and identification

Why does the text-based route fail so consistently? The problem is not lack of effort but a fundamental structural limitation. Consider the most challenging case for voice identification: the agent has ten enrolled voice samples on file, and a new recording arrives from a different room, on a different day, with different acoustics. It must determine which of the ten speakers it belongs to. An agent has four options (Table 1). Three of them discard the raw perceptual signal that distinguishes one voice from another. Only the fourth route preserves this signal — yet doing so immediately reveals a deeper issue: every approach implicitly conflates two distinct sub-problems. Recognizing a perception requires both (1) localizing the referent in a cluttered context and (2) encoding its identity. No single component performs both tasks well.

The three routes that discard the signal. *Caption-and-search* [Chhikara et al., 2025, Wang et al., 2024] writes the voice down (“a male voice, mid-30s, slight Eastern-European accent”) and later retrieves by similarity over the words. While the description is true, it is far too generic: it matches thousands of speakers, so cosine similarity over captions cannot reliably distinguish even ten users. *Recognize-and-label* [Long et al., 2025] calls a recognizer (ArcFace [Deng et al., 2019], ECAPA-TDNN [Desplanques et al., 2020]) and stores the label it returns (“speaker

Table 1. Four routes for remembering a perception, plus ours (citations in text). Only routes that keep the raw perception can discriminate across conditions. Only embedding retrieval and our method inherit the encoder's recall; ours additionally localizes the referent with a VLM and reads the identity back *inside* the model as a token the model conditions on, which is training-free and in $\mathcal{O}(1)$.

Route	What it stores	Keeps percept?	Cross-condition	Reg. cost
Caption-and-search	text description	no	chance	$\mathcal{O}(1)$
Recognize-and-label	a bare label	no	label only	$\mathcal{O}(1)$
Per-concept tuning	tuned token/adapter	yes	strong	gradient / identity
Embedding retrieval	raw embedding	yes	encoder-limited	$\mathcal{O}(1)$
Parametric (ours)	embedding + LM value	yes	**= encoder, in-model**	$\mathcal{O}(1)$

47″). The model can recall the assigned label, but it never stores the actual perception so it cannot compare a new voice against previous ones. *Per-concept tuning* [Nguyen et al., 2024, Alaluf et al., 2024] fine-tunes a dedicated token or adapter per concept. While effective, this approach is impractical. Each identity requires gradient updates, and the resulting artifact is concept-specific. This cost profile does not scale for a user who registers dozens of new people, voices, and objects.

The one that keeps the signal, and what it still lacks. *Embedding retrieval* bypasses text entirely: it indexes by the raw perceptual embeddings, and retrieves by cosine similarity. This is the only text-free route that preserves the original perception, and serves as our strongest baseline. It succeeds to the degree that the encoder is invariant across recording conditions, and fails when the invariant is broken. On a 2180-identity face pool, it reaches 0.95 recall at ten identities, but performance degrades as the pool grows. Crucially, this method assumes the perception has *already* been cleanly isolated—i.e., it receives a clean voiceprint or a tight face crop. In realistic cluttered scenes, this isolation step itself is extremely difficult. A bare encoder is helpless here: embedding an entire two-person photo yields only 0.05 accuracy at identifying the intended person (Section 5). This is the second problem that all four routes conflate: *localizing* the referent is not the same as *identifying* it. Even the strongest text-free route only solves the latter.

Our approach builds on this identity step while addressing its two key shortcomings. We place a VLM in front to perform grounding (resolving and locating the intended referent), and we give the resulting identity a new home inside the model. Instead of retrieving a neighbor from an external index, we store the encoder key together with a dedicated marker token and read it back via attention directly at the model's output head (Section 3). The recalled identity thus becomes a native token the model can condition on during generation. Recognition performance itself remains unchanged — our sharp read closely approximates a hard argmax over encoder cosine similarities — but the memory now additionally provides grounding, an in-model representation, $\mathcal{O}(1)$ registration cost, and seamless portability across frozen models.

3 Grounding perceptions in the model's own representation

We decompose perceptual recall into three steps, which we address in turn: (1) *ground* the referent in its visual or auditory context using a VLM, (2) *identify* it with a dedicated encoder, and (3) *store* the resulting identity as a token that can be read directly inside a frozen language model. The first two steps leverage off-the-shelf components placed at their points of strength.

The third step—ATTMEM, our core contribution—is the novel mechanism introduced in this paper.

Grounding: resolving the referent in context. Perceptions rarely arrive pre-cropped. When a user points to one person in a group photo, refers to the painting on the left, or mentions "the voice that just spoke," the system must first resolve this context-dependent reference into a specific region before encoding can begin. This is precisely what vision-language models excel at, and what specialist encoders cannot do: the VLM grounds the user's referent against the surrounding scene and the user's words, returning a localized region (a bounding box for images, or a temporal span for audio). We use the VLM's native grounding capability for this step. Where available, we further refine the region using a dedicated detector (e.g., RetinaFace [Deng et al., 2020] for faces, from the same InsightFace suite as ArcFace). This handles bounding box imprecision and unaligned scenes before identity encoding. The necessity of this grounding step is strongly supported by experiments: encoding an entire ungrounded scene recognizes the intended referent at only 0.05 recall for faces and 0.11 for paintings—close to random—because a single embedding cannot disambiguate the chosen referent from distractors (Section 5).

Identification: a dedicated encoder, not the model's own tokens. Once the referent has been grounded to a specific region, we extract its identity using a dedicated perceptual encoder. The resulting L2-normalized embedding serves as the identity *key*. We use ArcFace for faces, ECAPA-TDNN for speakers, CLIP for painting style, AST [Gong et al., 2021] for acoustic scenes, and a wav2vec2 emotion encoder [Baevski et al., 2020] for tone. It is tempting to skip this step and instead extract identity directly from the VLM's own context-conditioned representations (i.e., asking the model to "output the embedding of the face"). However, this does not work well in practice. A linear probe on the VLM's hidden states achieves only 0.25 recall at identifying the correct referent (raw hidden state 0.14), far below the oracle performance of 0.80. Similarly, the VLM's native vision tokens achieve only 0.54 recall on AgeDB cross-age faces, compared to 0.81 for ArcFace at the same scale N=20 (Figures 2, 10). This performance gap persists even in end-to-end tests inside a full VLM (§6). In short, VLMs are strong localizers but weak identity encoders. Strong identity extraction therefore requires a dedicated perceptual encoder kept separate from the model's internal representations.

Storage: an in-model attention memory. The identity key is stored and retrieved using ATTMEM, a table of key-value rows on a frozen language model (Figure 4). Each row i contains a *key* $\boldsymbol{k}_i$ (the encoder embedding above) and a *value* $\boldsymbol{v}_i$ (the language model's own embedding for a marker token assigned to that identity). Storing the model's native vector as the value (rather than an arbitrary learned one) is a deliberate choice. It makes the eventual logit boost for the right marker clean and self-reinforcing, instead of relying on an accidental alignment of two unrelated vectors.

Reading the memory is one step of attention. When a localized perception arrives at generation time, right before the output head, the model forms a query $\boldsymbol{q}$ from its encoder key and computes

$$\boldsymbol{w} = \mathrm{softmax}\big(\beta\, \boldsymbol{q}^\top \boldsymbol{K}\big), \qquad \boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V}, \qquad \boldsymbol{h}' = \boldsymbol{h} + g \cdot W_o\, \boldsymbol{r},$$

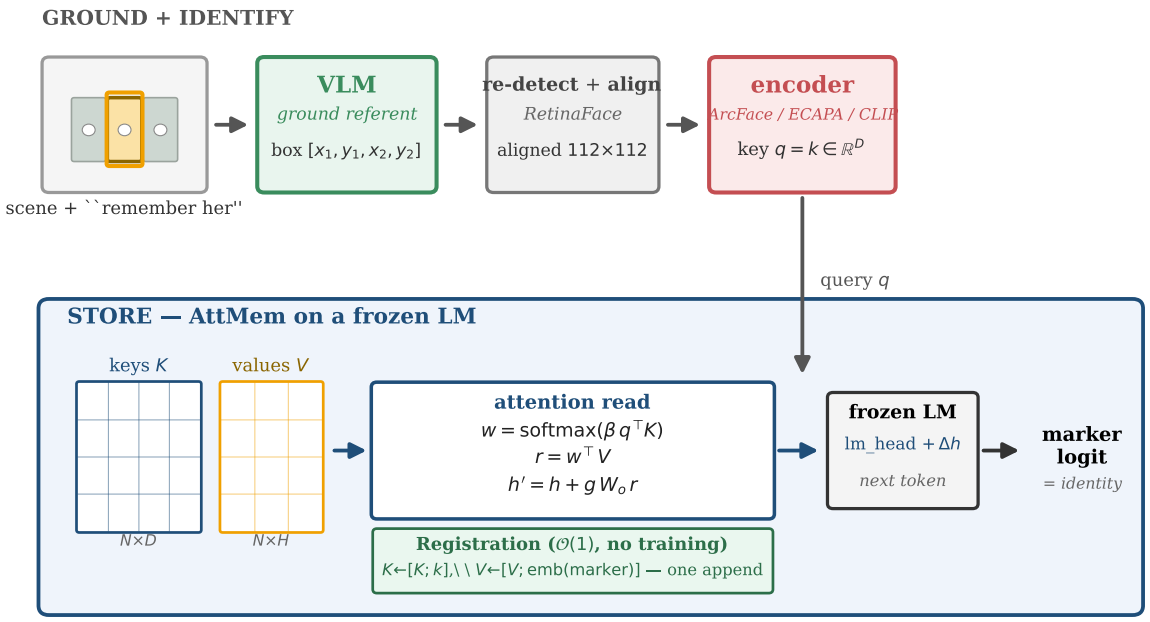


Figure 4. The grounded memory in detail. **Ground + identify (top):** the VLM resolves the referent to a box, a modality detector re-detects and aligns the region (RetinaFace for faces), and a frozen perceptual encoder (ArcFace / ECAPA / CLIP) maps it to an L2-normalized key $q{=}k \in \mathbb{R}^D$. **Store (bottom):** AttMem holds keys K ($N{\times}D$) and values V ($N{\times}H$, the model's own marker-token embeddings); the attention read $\boldsymbol{w}{=}\mathrm{softmax}(\beta\, \boldsymbol{q}^\top K)$, $\boldsymbol{r}{=}\boldsymbol{w}^\top V$, $\boldsymbol{h}'{=}\boldsymbol{h}{+}g\, W_o \boldsymbol{r}$ adds a residual at the output head that biases the next token toward the matching marker. The read has no trainable parameters (β, g are constants); registration is a single $\mathcal{O}(1)$ append.

where $\boldsymbol{K}, \boldsymbol{V}$ stack the bank's keys and values, β is the attention sharpness, g a residual gain, and W_o an optional output projection — the identity in all our experiments, kept in the equation only to mark where a projection would sit. In words, the perception softly looks up the bank, pulls back a blend of the matching markers' model-side vectors, and nudges the next-token prediction toward them. The blend is the entire read, and it adds *no trainable parameters*: β and g are two constants, and the values are the model's own marker embeddings.

Design choices for a faithful read. Four design decisions determine whether the read recovers the encoder's nearest neighbour cleanly with no training.

1. *Do not* divide the attention logits by $\sqrt{D}$: with L2-normalized keys, that flattens every cosine similarity into near-uniform attention.
2. Use a high sharpness β: this makes the attention a near-hard argmax over the bank rather than a diffuse average.
3. Attach the hook at the output head (not at an intermediate layer), so the residual reaches the logits undiluted.
4. Set a large gain g so the retrieved marker dominates the hidden state, producing an exact read. On models with untied output embeddings (whose output-embedding norms run smaller), an even larger g is required (Appendix A, Table 7).

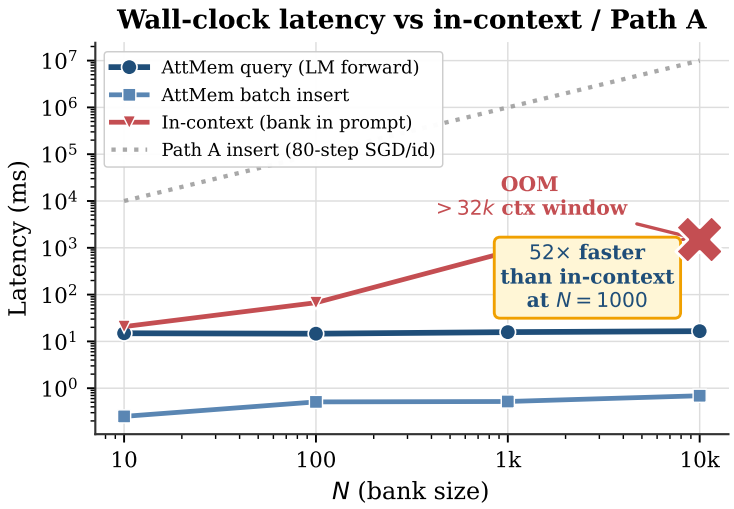


Figure 5. Recall latency remains near constant (∼15 ms) as bank size increases, as it is dominated by the frozen model's own forward pass. Insertion is $\mathcal{O}(1)$. Feeding the same registered perceptions in as context grows linearly and exhausts the 32k context window (Qwen2.5-3B) at ten thousand identities. Per-concept fine-tuning (PATH A-style) costs gradient steps per identity. Log–log axes.

With these four choices, the read reproduces the encoder's recall exactly (§5.4), and registering a user reduces to a single row append with no training required.

Registration versus fine-tuning. The cost of writing to memory is a critical practical consideration (Figure 5). Adding an identity is a single `torch.cat` operation (about half a millisecond to add a thousand), and recall is constant-time regardless of bank size, as the lookup is a small matrix multiply that is dwarfed by the model's own forward pass (∼15 ms either way). By contrast, the natural alternative of feeding all previously registered perceptions into the model as context grows linearly with the number of identities per query and quickly exhausts the context window. Our mechanism is 52× faster at a thousand identities, and is the only one that remains usable at ten thousand. A memory system intended for a companion agent—which must be updated after every conversation—must be inexpensive to write to.

4 PerceptMem: a cross-condition perceptual-memory benchmark

To measure perceptual recall we introduce **PerceptMem**, a benchmark spanning five sub-modalities. Each sub-modality was chosen because the discriminative signal is known to be severely degraded or destroyed when converted to text captions (Table 2). The benchmark covers both vision and audio, as well as identity and style, spanning physical appearance and affective cues:

The benchmark uses standard, widely adopted datasets and encoders: LFW [Huang et al., 2008] and AgeDB [Moschoglou et al., 2017] read with ArcFace [Deng et al., 2019], WikiArt [WikiArt, 2010] with mid-layer CLIP [Radford et al., 2021], LibriSpeech [Panayotov et al., 2015] with ECAPA-TDNN [Desplanques et al., 2020], ESC-50 [Piczak, 2015] with AST [Gong et al., 2021], and RAVDESS [Livingstone and Russo, 2018] with a wav2vec2 emotion encoder [Baevski et al., 2020].

PerceptMem is deliberately designed to evaluate *memory* rather than raw perception. Every

Table 2. The five PerceptMem sub-modalities. Each is included because no caption can carry its discriminating signal.

Sub-modality	Source / encoder	Why text cannot carry it
Face across age & lighting	LFW+AgeDB / ArcFace	"brown-haired man" fits thousands
Painter style	WikiArt / CLIP-mid	no caption separates early vs. late Monet
Speaker across recordings	LibriSpeech / ECAPA-TDNN	timbre is not transcribable
Acoustic scene	ESC-50 / AST	"traffic noise" fits every street
Tone of voice	RAVDESS / wav2vec2	"sounded tired" lacks a personal baseline

sub-modality provides a fixed, off-the-shelf encoder that all methods can use as black-box infrastructure, ensuring that no approach gains an advantage from using a stronger encoder. The only allowed operations are `register` and `recall`. All data is cross-session: each registered sample and its corresponding query come from different recordings. Training identities never overlap with evaluation identities. For a memory of size N, we register one sample per identity and then issue cross-condition queries, asking the method to retrieve the correct identity. Throughout this paper, *recall* refers to recall@1—the fraction of queries for which the top-scoring registered identity is correct. For retrieval-based methods this is the encoder's nearest neighbor; for our in-model mechanism it is the argmax over the registered marker token logits. For tone of voice, each registered "identity" is a (speaker, emotional state) pair. Successful recall therefore requires matching a paralinguistic state across different utterances—a capability that per-utterance captions cannot achieve without access to the user's personal history.

For statistical rigor we expand the five conceptual sub-modalities into a 12-domain, 1,080-task benchmark across multiple datasets and encoders. Each domain is defined as a (dataset, encoder) pair; their pool sizes and per-domain recall statistics are provided in Appendix Table 4 (see also Section 5 and Figure 10). We report a strong text-only baseline and a random guessing floor on every comparison. Our main reference point throughout is **embedding retrieval**—i.e., using the same encoder with cosine nearest-neighbor search over the registered keys. We refer to this as the *encoder ceiling*, since it represents the best possible performance achievable using the encoder's own similarity metric. Our in-model read is designed to reproduce this ceiling rather than surpass it.

5 Empirical evaluation

We structure our evaluation around the central claims of the grounded memory approach. First, we show that the performance drop from using text is both real and substantial: captions are fundamentally unable to carry perceptual identity (§5.1). Second, we demonstrate that grounding effectively recovers this signal. Combining VLM-based grounding with a dedicated encoder reaches the correct-region oracle in cluttered scenes (§5.2), and the same decomposition generalizes successfully to audio and video (§5.3). Third, we establish that the recognition core is the dedicated encoder itself, and that faithfully reading it in-model is a deliberate design strength, not a limitation (§5.4). Matching the encoder's performance is the goal: it shows we have preserved the encoder's quality while adding the benefits of grounding, in-model access, training-free registration, and portability across frozen models. We then show that the in-model store enables seamless composition with text memory (§5.5), matches the strongest pipeline baselines while significantly outperforming in-context image methods on both accuracy and

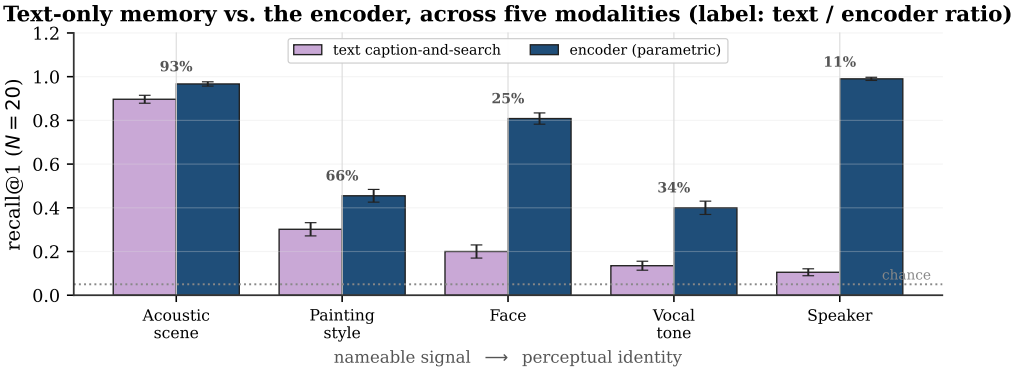


Figure 6. Text-only caption-and-search versus the parametric encoder, paired on identical draws across all five modalities (N=20, 30 draws, 95% CI); the label above each pair is the text/encoder ratio. Modalities are ordered by how nameable the signal is. Text nearly matches the encoder on the nameable acoustic scene (93%) and degrades toward random chance on perceptual identity (speaker 11%), tracing the router's boundary between the text store and the parametric bank.

cost (§5.6), and fully preserves the model's original text behavior (§5.7). Finally, we integrate all components into an end-to-end multi-session agent (§5.8).

5.1 Text captions cannot carry perceptual identity

A common baseline is to describe the perception in words, store the resulting text, and retrieve by semantic similarity. We evaluate this approach across *all five modalities*, giving it the strongest possible setup: an LLM (Qwen2.5-VL [Bai et al., 2025b] for images, Qwen2.5-Omni [Xu et al., 2025] for audio) generates a re-identification caption, which is then embedded with a sentence encoder. Recognition is performed via cosine nearest-neighbor search over these captions and compared directly against the dedicated encoder on the same samples. The results reveal a clear pattern (Figure 6). Caption-based recall approaches that of the dedicated encoder when the signal is easily *nameable* (e.g., an acoustic scene described as "someone typing on a keyboard"). However, performance drops sharply toward chance when the signal is a fine-grained *perceptual identity* (e.g., a specific voice, where descriptions such as "high-pitched with an accent" apply to many speakers). In short, text is a competent memory for concepts that can be described verbally, but a poor one for perceptual identity that cannot. This is exactly the distinction our capacity-law router exploits when deciding between the text store and the parametric memory bank (§8).

5.2 Grounding recovers oracle recognition in cluttered scenes

In deployment, perceptions rarely arrive pre-cropped. The referent usually appears among other people or objects, and the system must first resolve which one the user means before encoding can begin. An ablation study isolates the contribution of each component (Figure 7). A *text-only* memory, in which the VLM captions the referent from the full scene and retrieval is performed by text similarity, achieves only 0.12 recall on faces and 0.17 on paintings. A *store-only* memory that embeds the entire ungrounded scene performs even worse, close to random guessing, because a single embedding cannot disambiguate the intended referent among distractors.

In contrast, *grounding* the referent first and then identifying it with the encoder recovers the

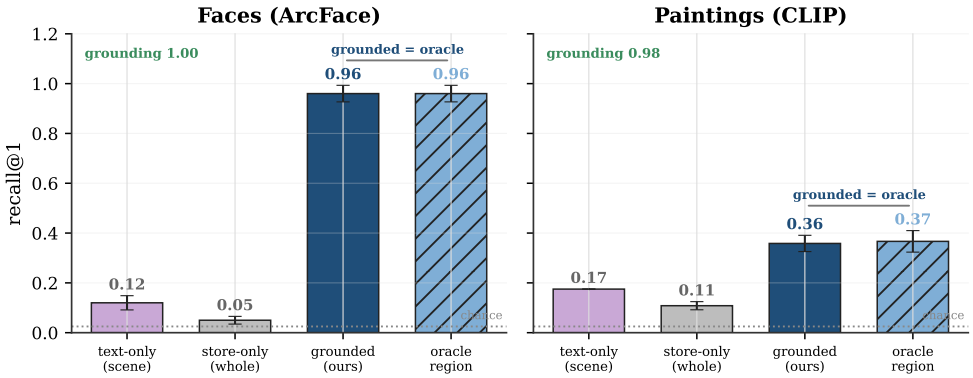


Figure 7. Ablation of the grounded memory on cluttered scenes (M=40, two referents per scene). Performance increases left to right: *text-only*, where the VLM describes the referent from the whole scene and the captions are matched; *store-only*, where the whole scene is embedded with no grounding; *grounded*, our ground + identify + store strategy; and the correct-region *oracle*. Both text-only and store-only perform near random guessing, while grounding reaches the oracle-level performance. Retrieval uses the encoder's nearest neighbor in all cases, so the large gain comes from grounding itself, not the matching method. Faces: ArcFace with landmark alignment, 5 seeds; paintings: CLIP with no alignment, 3 seeds. Error bars show ±95% CI.

correct-region oracle performance in both domains: 0.96 recall on faces and 0.36 on paintings (each matching its respective oracle). The relatively low ceiling on paintings is due to CLIP's limitations, not grounding. The VLM successfully localizes the correct region on nearly every trial. Since retrieval is performed using the same encoder nearest-neighbor in both cases, the large performance gain comes entirely from grounding, not from the matching mechanism.

The advantage of grounding is robust to increasing scene clutter and to different VLM scales (Figure 8). As the number of referents per scene grows from two to six, grounding accuracy remains perfect (1.00) up to K=5 and drops only marginally at K=6 (0.99). Grounded recall closely tracks the oracle (0.95–1.00) across all densities, while whole-scene embedding recall stays near the floor. In contrast, text-only recall degrades as clutter grows, falling from 0.12 at K=2 to 0.05 at K=3 as the caption must disambiguate more distractors. Grounding accuracy is already saturated with a 7B VLM and unchanged at 32B (1.00 in both cases; recall 0.960 vs 0.958). Overall, grounding is lossless and domain-general: the performance it enables is limited only by the quality of the downstream encoder—modest for CLIP-based style features and near-perfect for ArcFace-based identity.

5.3 Grounding generalizes to audio and video

The proposed decomposition is not limited to vision. The same three steps apply when the referent is a *speaker* in a multi-speaker audio recording or a *person* in a video (Figure 9). For **pure audio**, we concatenate K speakers from VoxCeleb [Nagrani et al., 2017] into a single clip and ask an audio-LLM (Qwen2.5-Omni) to localize the time span of the referenced speaker — the direct analog of visual bounding-box grounding. We then identify the speaker using ECAPA. Grounded recall matches the oracle at K=2 (1.00) and remains strong at K=3 (0.92 recall, 0.96 grounding accuracy). Performance drops at K=4 (0.67 recall, 0.69 grounding accuracy) as temporal localization of the fourth speaker becomes difficult, while encoding the entire clip is far weaker (recall 0.50 at K=2, falling to 0.26 at K=4) and the correct-segment oracle scores

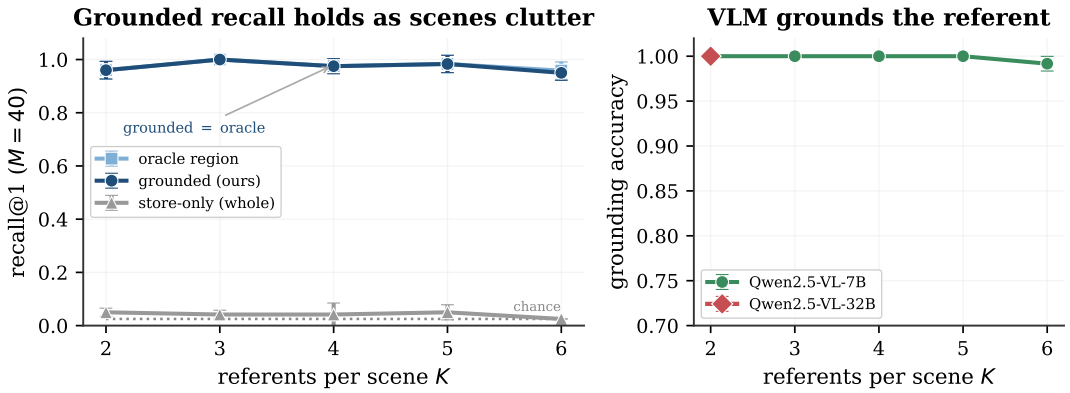


Figure 8. The grounded advantage is robust to scene clutter and VLM scale (faces, M=40). **Left:** As referents per scene grow from two to six, grounded recall (navy) closely tracks the correct-region oracle (light blue, nearly coincident), while whole-scene recall remains near the floor. **Right:** Grounding accuracy stays at 1.00 through K=5 and only drops slightly to 0.99 at K=6. Performance is identical for Qwen2.5-VL-7B and Qwen2.5-VL-32B, showing that the decomposition does not require a large grounding model.

1.00 throughout. This pattern holds in more realistic settings. On *VoxConverse* [Chung et al., 2020]—in-the-wild YouTube conversations with natural turn-taking, overlaps, and background noise—grounding is harder (grounding accuracy 0.75), but the grounded read still reaches 0.73 recall, nearly doubling whole-window encoding (0.38) toward the oracle (0.97) across 60 conversations. The *AMI* meeting corpus [Carletta et al., 2005] behaves the same way (grounded recall 0.75 vs. whole-window recall 0.34, grounding accuracy 0.88). These results mirror the vision findings: grounding, rather than identity encoding, is the component most affected by increasing complexity.

For **video** (combined vision and audio), we use real recordings from RAVDESS, in which the same 24 actors are captured across eight different emotions. This provides genuine paired (face, voice) data per identity. We register each actor from one clip (using an ArcFace face key and an ECAPA voice key) and evaluate on a held-out clip of a different emotion. Face recognition is near-saturated (1.00), while cross-emotion voice recognition is challenging (0.21). As a result, simple fusion of the two modalities with equal weight does not outperform face recognition alone, and we do not claim that it does. The real value of storing *both* modalities lies in robustness when one channel is corrupted—a capability that single-modality memory cannot provide (Figure 9, right). When the face is degraded to a distant, low-resolution view, face-only recall collapses to 0.09, but the fused memory maintains performance by relying on the voice (0.21). Conversely, when audio is corrupted by noise, voice-only recall drops to 0.16, but the fused memory falls back to the face (0.83). Fusion is the only representation that remains robust across degraded conditions—exactly the kind of real-world scenario a deployed companion agent must handle as people move off-camera or speak in noisy environments.

5.4 The in-model read reproduces the encoder's recall

Once the referent is localized and encoded, the memory bank is responsible for recognition. With a sharp read, the attention closely approximates a hard argmax over the same encoder cosine similarities used by embedding retrieval. As a result, its recall matches that of the encoder—neither better nor worse. A paired evaluation protocol, using identical registrations

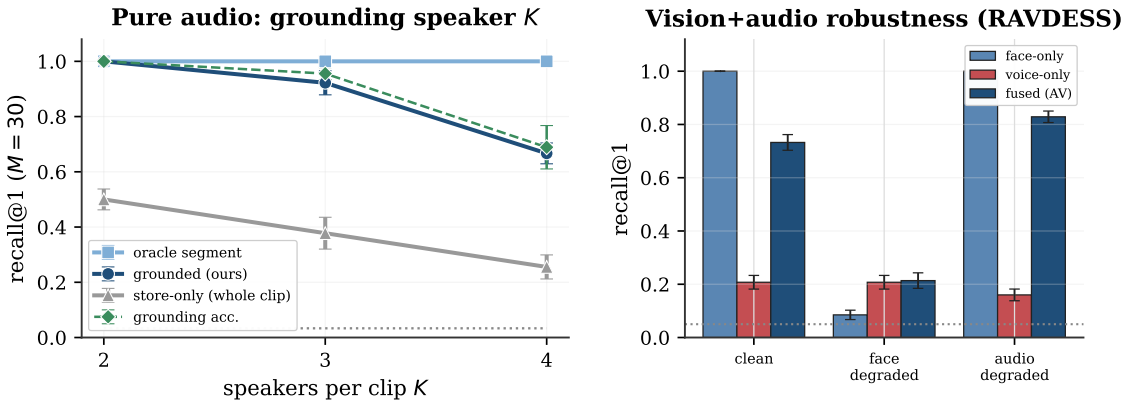


Figure 9. Grounding generalizes beyond vision. **Left — pure audio:** An audio-LLM grounds the K-th speaker's time span in a multi-speaker VoxCeleb clip and ECAPA identifies it (M=30, 3 seeds). Grounded recall (navy) closely tracks the oracle segment at K=2–3 and degrades at K=4 as temporal grounding becomes difficult, while whole-clip encoding stays well below the grounded read. **Right — vision+audio (RAVDESS video):** Each identity is represented by both a face and a voice key. Storing both modalities provides robustness to single-channel corruption: face-only recall collapses under distant/low-resolution views, voice-only performance is weak and further degrades under noise, but the fused memory consistently falls back to the surviving channel (M=20, 40 draws).

and queries with randomized target slots, confirms this equivalence across bank sizes from N=5 to N=1,000 ($|\Delta| \leq 0.001$ at each N, 20 draws; Table 5, Figure 17a). The in-model memory ties embedding retrieval on both random banks and adversarial banks of near-identical look-alikes. A learned similarity metric trained on the same features performs similarly (Appendix A). This tight match depends on using a sharp read. When the read is under-sharpened, performance begins to lag behind cosine similarity at $N \geq 300$ (Appendix A). The design choices described in Section 3 eliminate this gap.

Breadth and statistical rigor. At scale, the recognition core remains strong across all 12 domains and five modalities — a total of 1,080 tasks ($N \in \{10, 20, 40\}$, 30 draws, 95% CIs)— substantially outperforming random chance in every case (Figure 10). Importantly, recall *tracks the dedicated encoder*: it is near-perfect where the encoder is strong (e.g., speaker and face identity), and lower where the encoder is weaker (e.g., style and tone). Notably, the VLM's own native vision tokens perform substantially worse than a purpose-built encoder on the same AgeDB faces. This provides clear evidence that the encoder — not the memory mechanism — determines the performance ceiling in each domain (§6).

Training-free on any frozen model. Because the read operation is a similarity-weighted blend of marker tokens, recall does not depend on the specific host language model and requires no training to transfer. We verify this across ten model families (Qwen2.5 1.5B/7B [Yang et al., 2024], Qwen3 4B/8B/14B [Yang et al., 2025], Phi-3.5 [Abdin et al., 2024], SmolLM2 [Ben Allal et al., 2025], DeepSeek-Llama-8B [DeepSeek-AI, 2025], Mistral-7B [Jiang et al., 2023], and a hybrid-Mamba Granite [IBM Granite Team, 2025]). In all cases, the in-model read reproduces the encoder's recall (Appendix Figure 16). A five-architecture by five-modality grid shows the match holds to within one percentage point in every cell (Appendix Table 6). On models with tied embeddings the agreement is within 0.003; untied-embedding models match equally

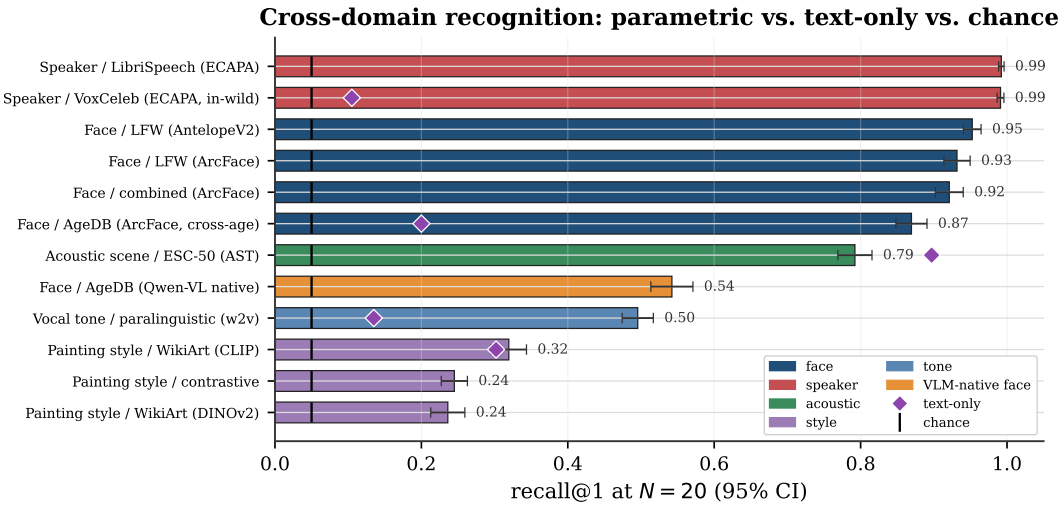


Figure 10. Recognition performance across 12 perceptual domains and 5 modalities (1,080 tasks; recall@1 at N=20, 30 draws, 95% CI). Results are colored by modality and compared against random chance ($1/N$, black ticks) and the text-only caption baseline where available (purple diamonds). The parametric memory closely tracks the dedicated encoder and substantially exceeds chance in all domains, far outperforming text captions on perceptual identity tasks (speaker, face, tone). The only inversion occurs for nameable acoustic scenes, where a text caption (e.g., "typing on a keyboard") slightly outperforms the weaker AST encoder — precisely the boundary used by our router (§8). Text works well for nameable categories, while the parametric bank is essential for fine-grained identity. For reference, the VLM's own native vision tokens (orange) perform substantially worse than a purpose-built encoder on AgeDB faces.

well with a modestly larger read-strength constant. Even the hybrid-Mamba architecture performs within one point. This universality is not the main contribution but a valuable supporting property: the storage mechanism can be hosted on *any* model the agent already uses, at essentially zero additional cost.

Rejecting strangers and verifying identities. A practical user memory must do more than closed-set recall, where the query is guaranteed to belong to a registered identity. It must also reject strangers it has never seen and accurately verify whether two perceptions belong to the same person. Both capabilities emerge naturally from the same read operation (cosine similarity over encoder keys). We evaluate these using standard operating-point metrics over 30 draws (Figure 11). On verification, the memory achieves strong separation between same- and different-identity pairs, with AUROC of 0.99 on voice and 0.97–0.99 on faces (equal error rates of 1.3% and 5–8% respectively). On open-set identification, it rejects never-enrolled strangers with 0.96–0.99 accuracy while correctly identifying enrolled individuals. As in previous experiments, these results closely track the underlying encoder: performance is high where the encoder is strong (voice and face) and lower where it is weak (e.g., painting style, open-set AUROC 0.54). The key takeaway is not new state-of-the-art numbers, but that our memory naturally supports the open-set recognition and verification modes required by a deployed companion agent, inheriting the encoder's separability in both settings.

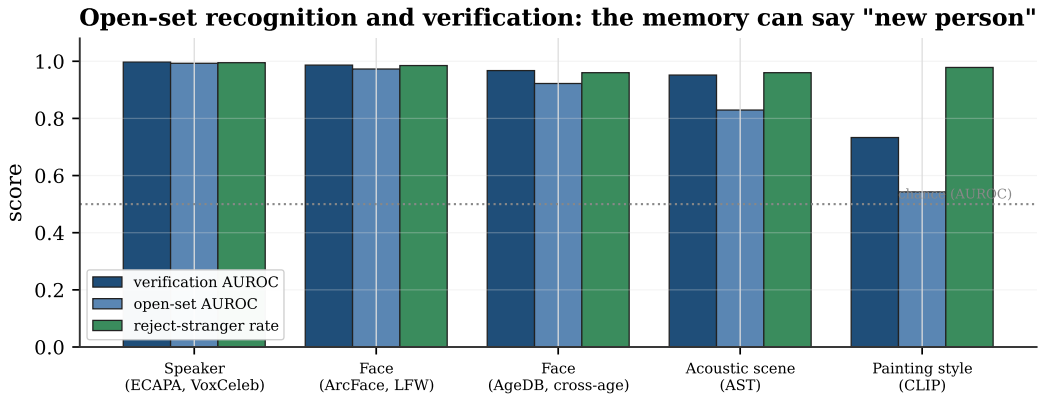


Figure 11. Beyond closed-set recall, the memory supports verification and open-set identification (N=20 enrolled, 30 draws). For each modality we report verification AUROC (same vs different identity), open-set AUROC (enrolled vs never-enrolled probes), and the reject-stranger rate at the balanced threshold. Performance is high where the encoder is strong (voice, face) and lower where it is weak (painting style), mirroring the same trend observed in recall.

5.5 Composition with in-context text memory

The key advantage of recalling a perception as a native token (rather than as an external label) is that the token can participate directly in the model's reasoning. We demonstrate this by registering M faces with dedicated name markers, then binding one fact per name in context (e.g., "Anna is a teacher"). We then show a held-out cross-condition photo and ask the model to retrieve the associated fact. The entire chain runs in a single forward pass: the face recalls its name through AttMem, and the name retrieves the corresponding fact from context. End-to-end accuracy improves 4–10$\times$ over the face-withheld chance baseline and closely tracks the product of recognition accuracy and the model's in-context lookup reliability (Figure 13). As M grows, the bottleneck shifts from the memory to in-context lookup. Figure 12 visualizes a single read: a sharp temperature concentrates the encoder's cosine similarities into a near-one-hot diagonal, producing decisive marker logits that match what pure encoder retrieval would select.

5.6 Comparison with retrieval and in-context baselines

The natural baseline is a retrieve-and-reprompt pipeline: cosine-retrieve the nearest registered face, recover its name, and re-prompt the model with the result. Because both approaches use the same encoder for recognition and the same frozen model for reasoning, they achieve *identical accuracy* on both the composition task and a multi-perception question such as "how many distinct people are here?" (Table 3). On pure recognition cost, the pipeline is actually cheaper, as it can count distinct identities using only cosine similarity with no model forward pass. We do not claim that the parametric memory is faster or more accurate at recognition itself. Its advantages are architectural.

The pipeline is a two-pass system: an orchestrator must decide *before* generation that recognition is needed, query an external index, and re-prompt the model. In contrast, the in-model read requires no such upfront decision. The memory bank is consulted automatically at every perceptual mention during the agent's normal generation. This enables seamless composition with ongoing reasoning (§5.5), supports interleaving multiple perceptions in a

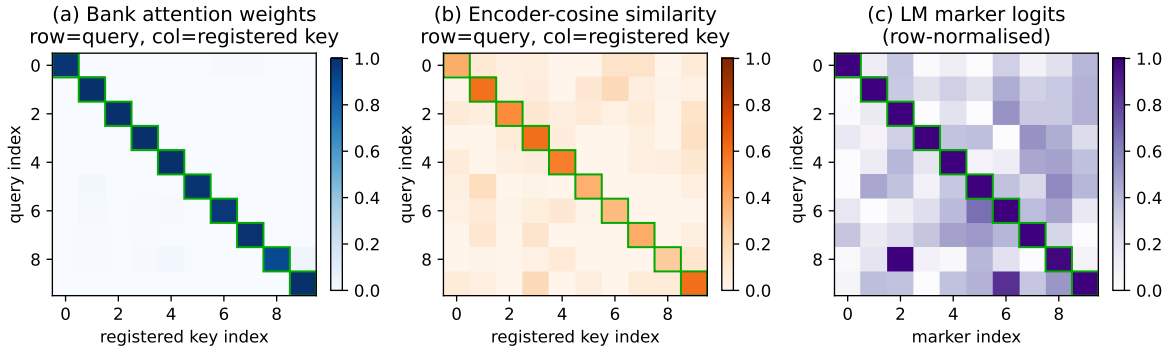


Figure 12. One recall, three views (held-out faces, ten identities). **(a)** With a sharp temperature, attention over the memory bank forms a near-pure diagonal (average 0.98), strongly committing to the top encoder match. **(b)** The underlying encoder cosine similarities show a softer diagonal (avg. 0.46) with noticeable off-diagonal mass; the memory faithfully inherits these similarities and their errors. **(c)** The resulting marker logits keep the correct identity at the top. Overall, the attention faithfully reads the encoder's nearest neighbour as a token; it does not sharpen the similarity.

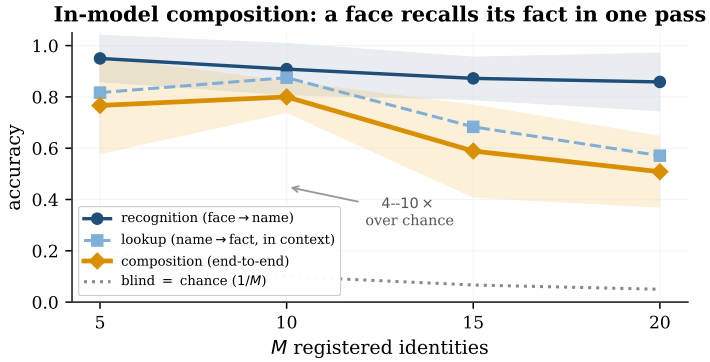


Figure 13. In-model composition: we register M faces with dedicated name markers, bind one fact per name in context, then query the associated fact using a held-out cross-condition photo, all in one forward pass. End-to-end accuracy (gold) closely tracks the product of recognition accuracy (navy) times the model's in-context lookup accuracy (blue) and beats the face-withheld random chance baseline by 4–10×. Results averaged over 12 draws; bands show ± one standard deviation.

single stream, and ships as a single frozen-model artifact with no external index to manage, synchronize, or secure. When a deployment can reliably afford the extra pass, the pipeline is a reasonable choice. The parametric in-model memory is preferable for agents that cannot know in advance when a perception will require identification.

The comparison becomes even sharper against the baseline a VLM can actually achieve in practice: keeping the perception directly in context as image tokens. A single face crop consumes 64 vision tokens in Qwen2.5-VL at 224px resolution, and 256–576 tokens at higher resolutions (a full image approaches a thousand). In contrast, AttMem represents the same perception using just one marker token—a 64–576× reduction in context tokens per perception. Because the memory bank lives outside the context window, it scales without bound, whereas keeping images in context exhausts even a 128k window after roughly two thousand crops. On accuracy, the in-context image baseline is also substantially weaker: the VLM relies on its general-purpose vision tokens (0.50 end-to-end accuracy, Appendix Table 11), far below a

dedicated face encoder (0.95). Thus, the parametric memory matches the performance of the strong external-index pipeline while significantly outperforming the in-context image baseline on both accuracy and cost. Combined with the capacity-law router, this is the core contribution of our approach.

Table 3. In-model memory vs. a retrieve-and-reprompt pipeline (Qwen2.5-3B, 10 draws). Accuracy is identical in all cases, since both approaches use the same encoder cosine similarity for recognition. The pipeline is cheaper for pure recognition tasks (e.g., counting distinct identities requires no model forward pass). The in-model read, by contrast, folds recognition directly into the model's own forward pass, eliminating the need for an external index.

Task	In-model acc	Pipeline acc	In-model lat.	Pipeline lat.
Compose (face→fact)	0.81	0.81	122 ms	98 ms
Count distinct, K=2	1.00	1.00	238 ms	0.09 ms
Count distinct, K=3	1.00	1.00	361 ms	0.09 ms
Count distinct, K=4	0.70	0.70	472 ms	0.09 ms

5.7 Preservation of text-only behaviour

The perceptual memory is designed to sit alongside—rather than interfere with—the model's ordinary text memory. The attention hook adds a residual only at perceptual positions, so the model's text-only behavior remains unchanged even when the memory bank is installed and populated. Concretely, with a populated bank the top-1 next-token predictions match the untouched model on every probe. The populated bank produces exactly the same logits as an empty one, and with the hook disabled the logits are bitwise identical (maximum $|\Delta\text{logit}| = 0$). The small residual deviation (≈ 0.375) observed on the wrapped forward pass stems from the host model's `inputs_embeds` numerics and appears regardless of whether the bank is empty or full. Registering faces and voices together shows the two banks remain fully independent with no training (cross-modal leakage of 0.017 for faces and 0.067 for voices). An agent can therefore store factual knowledge such as "my favourite restaurant is in Paris" in its text memory and perceptual identity such as "how Bibi looks across lighting and age" in its perceptual bank without compromising either. For contrast, we also evaluated a discrete codebook design (similar to previous PATH A-style approaches) that maps each perception to one of K learned codes. As expected, it never exceeded $\sim$0.07 recall beyond a few hundred identities—the same fundamental limitation as captioning—because it discards fine-grained signal that the continuous encoder preserves. These results are included in Appendix D.

5.8 An end-to-end multi-session agent

The components come together in a complete working agent. We construct a system consisting of a frozen language model equipped with the ATTMEM perceptual memory bank and a separate text store for facts. Users are enrolled across sessions, each with a face (registered from one photo) and an associated fact. When a returning user appears in a new cross-condition photo, the agent must identify them, answer a question about them, while also correctly rejecting anyone it has never met. We evaluate performance as the enrolled population grows from 10 to 80 users, averaged over five seeds (Figure 14). The perceptual capabilities scale gracefully: identification accuracy remains at 0.86 and stranger rejection at 0.81 even with 80

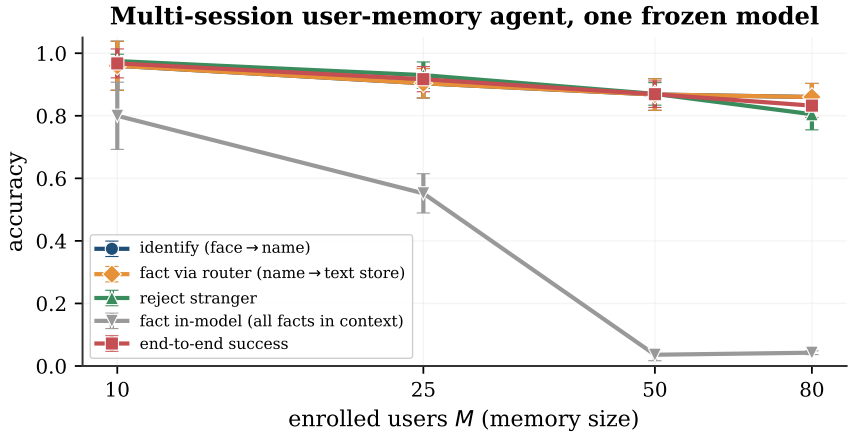


Figure 14. An end-to-end multi-session user-memory agent (Qwen2.5-3B, 5 seeds) as the enrolled population grows from 10 to 80 users. The system identifies returning users (face→name), recalls associated facts via the router (name→text store), and correctly rejects strangers. All capabilities scale gracefully, tracking the underlying encoder. In contrast, attempting to recall facts *in-model* by binding all of them in context collapses beyond a few dozen users — limited by the host model's context window, not the memory itself. This demonstrates why the router keeps exact facts in a text store. Overall end-to-end task success remains strong at 0.83 with eighty enrolled users, all within a single frozen model.

enrolled users, closely tracking the encoder (as observed throughout). Note that the higher rejection rates reported in §5.4 (0.96–0.99) were measured at only 20 enrolled users; as the number of identities increases, the operating point naturally tightens in line with the encoder's separability.

Fact recall highlights the value of the router. When perceptual identity is routed to the parametric bank (which recalls the name) and the name then keys an exact lookup in the text store, accuracy tracks identification performance (0.86 at 80 users). In contrast, attempting to bind all facts in the prompt and resolve everything in a single *in-model* pass collapses beyond a few dozen users (0.55 at 25 users, 0.04 at 50 users), because even a 3B model cannot reliably hold that many name–fact bindings in context. This provides a concrete demonstration of the router thesis: perceptual identity belongs in the parametric bank, where it scales effectively, while exact facts belong in a text store, where lookup remains precise. The resulting production path—perceptual bank for *who*, text store for *what*—achieves strong end-to-end task success (0.83 at 80 users) entirely within a single frozen model, with no external index required.

6 The encoder determines recall

Because the in-model read faithfully reproduces the encoder's recall, the overall quality of the memory system is determined by the quality of the encoder. On the exact same identities, recall tracks encoder strength closely: on AgeDB, ArcFace reaches 0.95 recall at ten identities and 0.75 at a hundred, whereas the VLM's own vision tokens manage only 0.64 and 0.34 respectively on those same faces. A learned similarity metric trained on the same features performs no better (Appendix A). This is the fundamental reason we use a dedicated encoder for the identification step rather than relying on the model's native tokens. The performance gap persists even when running the full system end-to-end inside a real VLM (§5.6, Appendix Table 11). In contrast,

the storage mechanism itself is model-agnostic: the read operation contains no encoder-specific machinery and ports to any frozen language model without training (§5.4). If better perceptual recall is needed for a task, the only effective solution is a stronger encoder — not a different language model or a more sophisticated read mechanism.

7 Related work

Text-first agent memory. The conversational-memory line — Mem0 [Chhikara et al., 2025], MemGPT and its Letta successor [Packer et al., 2023, Letta Team, 2024], MemoryLLM [Wang et al., 2024], M3-Agent [Long et al., 2025], benchmarked by LongMemEval [Wu et al., 2025] — stores transcripts and captions in a text index and retrieves by similarity. These systems define the captionable half of user memory that ours composes with. None of them keep a perception in its native modality, which is the missing half this paper adds (§5.1 measures what that omission costs).

Personalized vision-language models. MyVLM [Alaluf et al., 2024], Yo'LLaVA [Nguyen et al., 2024], MC-LLaVA [An et al., 2024], RAP [Hao et al., 2025], Online-PVLM [Bai et al., 2025a], and TAME [Hong et al., 2026] personalize a VLM to user-specific concepts, typically by learning an artifact per concept — a classifier head, a soft token, a projection. Our setting differs in both cost profile and mechanism: registration is a training-free $\mathcal{O}(1)$ row append. As shown in Appendix Table 10, these learned per-concept artifacts do not outperform the encoder's own cosine similarity on cross-condition identity tasks, while incurring per-identity gradient updates.

Retrieval inside the model. ATTMEM adopts the general shape of attention over an external bank from kNN-LM [Khandelwal et al., 2020], Memorizing Transformers [Wu et al., 2022], and RETRO [Borgeaud et al., 2022], which retrieve from token or chunk stores to improve language modeling. We repurpose this mechanism for a different goal: the keys are perceptual encoder embeddings rather than text hidden states, the bank stores a user's persistent perceptions rather than a corpus, and the read lands at the output head of a frozen model with no training.

Perception-to-token bridges and speaker recognition. Models such as Flamingo's resampler [Alayrac et al., 2022] and BLIP-2's Q-Former [Li et al., 2023] learn to project perception into a language model's token space. In contrast, our approach injects a single marker token whose value is the model's *own* embedding, which is what enables training-free operation. The speaker recognition and diarization literature [Nagrani et al., 2017, Desplanques et al., 2020, Park et al., 2022] treats "who spoke when" as a standalone pipeline that returns labels. We instead use the same encoders to produce grounded identity keys that the model can directly condition on, rather than treating them as external verdicts.

8 Discussion

The claim is structural, and the numbers illustrate it. Our central argument is not that "our method scores higher." Rather, it is that text is the wrong container for a large and important portion of what an agent should remember about a person, that recognizing a perception naturally decomposes into two distinct problems—grounding the referent in context and identifying it—and that a frozen model fronted by a VLM and backed by a dedicated encoder bank can solve both halves, whereas text and a bare encoder each solve only one (or neither).

The empirical results substantiate each piece of this argument: the performance penalty of text is real and strongly modality-dependent (Figure 6); grounding is both necessary and highly effective at recovering oracle-level recognition in cluttered scenes (Figure 7); and the recognition core is the dedicated encoder itself, given an in-model home that is training-free and model-agnostic (Appendix Figure 16). The core contribution is therefore the grounded, in-model perceptual memory container—not a particular recall number. The numbers we report simply reflect the quality of the underlying encoder.

This is additive infrastructure, not a replacement. The perceptual memory is designed to bolt onto the text-based memory systems that agents already use [Chhikara et al., 2025, Wang et al., 2024, Packer et al., 2023, Letta Team, 2024]. The strong non-regression results support this integration: text-only behavior is preserved exactly, and the perceptual bank activates only at perceptual mentions. As long-term memory benchmarks such as LongMemEval [Wu et al., 2025] begin to incorporate perceptual content, the structural gap highlighted in this work will become directly measurable.

The captionable half is going parametric too. This paper is one half of a broader shift: a personalized agent should keep its memory *inside* the model rather than in an external index. A parallel line of work is making the same architectural choice for the *captionable* (textual/factual) half of user memory. *User as Engram* [Li, 2026c] stores per-user facts in a hash-keyed parametric memory and shows that the intuitive alternative—a per-user LoRA adapter [Hu et al., 2022]—is *reasoning-negative*: it successfully memorizes facts but degrades the model's general reasoning, because LoRA edits are global while row insertions are local. *User as Code* [Li, 2026b] encodes user information as executable typed state, and *Programmable KV Cache* [Li, 2026a] maintains editable, composable notes directly in the attention cache.

Our perceptual memory shares that thesis with a different mechanism: continuous cross-attention over encoder banks rather than hash-keyed rows. This continuous channel, which carries a perception as a vector projected into the model's own embedding space (rather than as text), aligns with the approach taken by *The Latent Bridge* [Li and Shi, 2026] for inter-model communication, where a learned latent link outperforms writing and re-reading text. Together, the two halves compose into one memory system that holds both what the user said and what they are like.

The router: what a latent memory can and cannot hold. The split between the parametric perceptual bank and the text store is not an arbitrary convention. It follows from what we measured when compressing content into k soft tokens read by a frozen language model (Figure 15). Perceptual identity is capacity-limited and degrades gracefully. Compressing M registered identities into k prototype slots and querying with a cross-condition sample, the system can distinguish roughly a k/M fraction of them. This holds consistently across faces, voices, acoustic scenes, painting styles, and tones of voice (Appendix Table 8). With one slot per identity, performance matches the encoder; with fewer slots, identities are lost proportionally. The general law is recall $\approx \min(1, k/M) \cdot C(M)$, where $C(M)$ is the encoder's own recall at M registrations. The slot-compression factor $\min(1, k/M)$ is universal, while the ceiling $C(M)$ is determined by the encoder (near 1 for strong encoders such as face and voice, lower for weaker ones such as style and tone).

This behavior is not an artifact of k-means clustering: a compressor whose k slots are learned via gradient descent follows the same curve, as k hard slots can separate at most k out of M identities (a pigeonhole bound).

Exact factual content fails in a different way. Here a small write head (16 learned queries that cross-attend to the frozen model's middle-layer hidden states) compresses a document of M lines of the form `Name: CODE` into k=16 soft tokens, and a LoRA reader must answer `Name:` with that name's six-character code. A code counts only if every character is right. In our original runs (3,000 training steps), the latent held one code (0.87) but collapsed at two (0.06, and near zero beyond). At that budget more latent tokens did not help either, but we have not repeated that k sweep with longer training. A follow-up study on two seeds (Appendix C) shows that the collapse is mainly undertraining, not a hard limit. Rerunning the original protocol on three seeds reproduces it: two-code recall is 0.00–0.04. Trained to a plateau with nothing else changed, the same model recalls two codes at 0.555 and 0.400 (seeds 42 and 43) with the original LoRA rank of 32. One seed needs more than five times the original budget to pass 0.5; the other never reaches it. With rank 16 it reaches 0.715 and 0.711, in fewer steps. Storage is not the bottleneck. Linear probes read either code out of a two-code latent at 0.79–0.90 per-character accuracy, close to the 0.92–0.94 for a single code. Many errors return the *other* stored code intact: 14–16% of all answers at rank 32 and 26–28% at rank 16, where almost every error is such a swap. The hard part is *binding*: pairing the queried name with the right one of the codes held in the same tokens. Binding gets harder with more items. With three codes, the shared latent reaches only 0.135 and 0.230 at rank 32 (0.311 and 0.410 at rank 16). Longer codes make it worse: on one seed at rank 32, three codes fall from 0.604 with two-character codes to 0.088 with eight-character ones, and at six to eight characters the latent also loses content, with only 14–21% of answers being any stored code. Giving each item its own slots helps a lot but does not settle it. A hard attention mask that lets each group of latent queries read only one line of the document binds two codes at 0.893 and 0.998, at the same k=16. With three codes it reaches 0.992 on one seed but only 0.639 on the other, where the latent is cleanly slotted and yet 31% of answers are another stored code, intact. Separate slots keep the items apart in storage; the reader must still learn which slot belongs to which name. The mask also needs the line boundaries when the memory is written; the shared latent does not.

What a latent can do with exact facts therefore depends on keeping items apart, as for perceptual identity, but slots alone are not enough. In some runs a shared latent splits its tokens, without being told to, into copies of one item or the other: both rank-16 two-code models did, and so did a rank-32 model with eight-character codes, so this is not a property of one rank. These split latents were among the best shared two-code models, and 96–99% of their answers were one of the stored codes, intact; their remaining errors were swaps (24–28% of answers). Latents that stayed mixed both corrupted codes and swapped them. Per-item slots, imposed by a mask, raised recall above the shared latent in all four runs, to near-perfect in three. Separating items fixes storage; attaching the queried name to the right slot is the error that remains. That, rather than an impossibility result, is the case for the router. A text store does exact lookup with no training, no slot assignment and no name-to-slot reading to learn. A shared latent needs several times our original training budget to bind even two codes, varies widely from seed to seed, and still swaps them. Per-item slots need the writer to know where each fact begins and ends, which is the segmentation a text store gets for free, and even then the reader can attach a stored code to the wrong name. Perceptual identity, by contrast, cannot be written down without loss (Figure 6), and the parametric bank already gives it one row per identity with no training. We therefore keep the captionable half in a text store and the

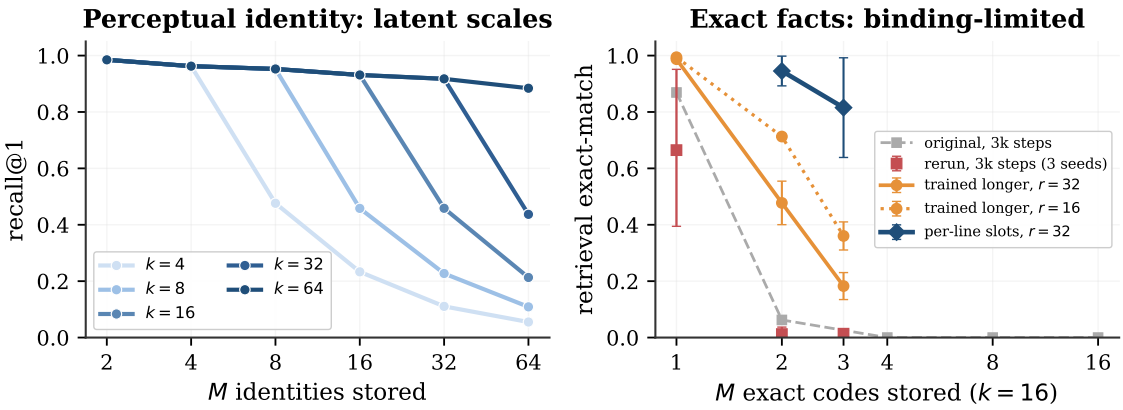


Figure 15. What memory compressed into k soft tokens read by a frozen language model can hold, the measurement behind the router. **Left:** Perceptual identity is capacity-limited. When compressing M faces into k prototype slots and querying with a cross-condition sample, recall scales as $\approx \min(1, k/M) \times C(M)$ where $C(M)$ is the encoder ceiling (≈ 1 for faces). Voices follow the same pattern. One slot per identity recovers full encoder performance, and more slots allow more identities to be stored. **Right:** Exact factual content (M six-character codes in k=16 tokens) is limited by binding. At our original budget of 3,000 training steps the latent holds one code (0.87) but collapses at two (0.06; grey). A rerun of that protocol on three seeds reproduces the collapse (red; bars span the seeds). Trained to a plateau, the shared latent recalls two codes at 0.40–0.56 (LoRA rank 32) or 0.71–0.72 (rank 16) but falls off at three (orange). A hard per-line write mask, which gives each item its own slots, does better still but not reliably: three codes at 0.99 on one seed and 0.64 on the other (navy). Orange and navy points are the mean of seeds 42 and 43, with bars spanning the two (Appendix C).

perceptual half in the parametric bank. Whether a slotted latent could hold exact facts reliably in practice is open: our evidence covers two seeds (one for code lengths other than six) and at most three codes of up to eight characters, one per line.

Privacy and responsible use. A perceptual user memory stores biometric templates (faceprints and voiceprints) and must be held to the high standards required for such data. Enrollment should be consented and explicit (the "remember her" act of Figure 3 is an affirmative request, not passive collection). The bank should be stored on-device or encrypted at rest, and the system must not be repurposed to identify individuals who have not consented to being remembered. Two properties of our design support responsible deployment. First, because each identity occupies a single row, deletion is as simple as registration: removing the row completely erases the identity with no fine-tuned weights to unlearn. This provides a clean implementation of the right to be forgotten, which per-concept tuning approaches cannot easily offer. Second, because recall performance is determined by the underlying encoder, the memory inherits the encoder's known demographic biases (particularly in face recognition). Deployments should therefore carefully evaluate the chosen encoder on their target population and favor verification-style operating points with strong stranger rejection enabled (Figure 11), ensuring that unenrolled individuals are refused rather than misassigned to an enrolled identity.

Limitations.

1. **The memory cannot exceed the encoder.** Its recall matches that of the underlying encoder — paired to within ± 0.001 from N=5 to N=1000 (Table 5, Figure 17a). Consequently, performance at very large scale declines exactly as the encoder's cosine similarity does

(0.78 at a thousand faces), and no in-model mechanism can recover more.

2. **Scale of cross-condition evaluation.** Our largest cross-condition pool holds 2,180 real identities, from which we measure recall to N=1000 registered users. Constant-time behaviour is confirmed up to ten thousand rows, but recall at that scale remains to be tested with larger datasets.

3. **Grounding robustness.** The grounding step inherits the referring-expression resolution capabilities of the VLM. While it is near-lossless on well-separated composites (grounding accuracy 1.00 up to K=5), performance degrades under more realistic stress conditions. For example, jittered and rescaled faces pasted onto photographic backgrounds reduce grounding accuracy to 0.63–0.81 and end-to-end recall well below the oracle (Appendix Table 12). Audio grounding of the fourth speaker drops to 0.69. Denser, occluded, or noisier scenes will stress it further, making detector and diarization robustness—rather than the encoder—the practical bottleneck.

4. **Dependence on external encoders.** Inside a VLM, native vision tokens are weak identity keys (0.50 vs. ArcFace's 0.95 at N=10, Table 11). A dedicated, external modality-specific encoder is therefore required.

5. **Per-model hyperparameter.** The one per-model constant (read-strength gain) is set by inspection. Untied-embedding models need a larger value, which we have not automated.

6. **Binding evidence rests on two seeds, and the seeds disagree.** The 3,000-step rerun of the exact-fact experiment uses three seeds; every longer-trained and per-line-mask result uses two (Appendix C), and they spread widely: 0.555 vs. 0.400 for two codes at rank 32, and 0.992 vs. 0.639 for three codes with the per-line mask. Rank 16 beat rank 32 on both seeds, and on both its two-code latent split into item-specific tokens; one rank-32 run, with eight-character codes, split the same way. We read the split, not the rank, as what goes with good binding, but that rests on three split latents, not an established mechanism; the seed-42 rank-16 runs also came from an earlier revision of the training script with a different stopping rule. The per-line mask needs item boundaries when the memory is written. We have not tested it on free-form text or beyond three items, the code-length runs (two to eight characters) have one seed, so each cell could move by about 0.15, and the k sweep ("more latent tokens do not help") was measured only at 3,000 steps and has not been repeated with longer training.

7. **Baselines.** The closest prior systems (MyVLM [Alaluf et al., 2024], Online-PVLM [Bai et al., 2025a]) have released neither code nor checkpoints. Comparisons are therefore made against charitable reimplementations of their core mechanisms (Appendix A, Table 10).

9 Conclusion

A photograph and a written description of a face are not two encodings of the same thing. The description has already discarded much of what enables recognition. Today's agent memory systems keep only the description. We have argued that they must also keep the photograph

itself—and that doing so effectively requires grounding. A vision-language model resolves the referent the user means in context. A dedicated encoder turns that referent into an identity that survives variations text cannot capture. A frozen language model then stores this identity as a content-addressable row, read directly inside its own forward pass. Each component addresses what the others cannot: the VLM cannot reliably identify, the encoder cannot ground, and an external index cannot return the answer as a native token inside the model. Together, they achieve oracle-level recognition that none of them can deliver alone. The recognition performance itself belongs to the encoder; our contribution is giving it an effective home—training-free, constant-time, and portable to any frozen model. The two memory systems are complementary: text for what the user *said*, and parametric multimodal memory for what the user *is like*. An agent equipped with both strictly dominates one that has only text. This is where a face stops being reduced to a sentence, and starts becoming a memory.

Acknowledgements

We thank Xiangyu Zhang from StepFun AI for the initial ideas that inspired this paper; he introduced the problem of multimodal user memory to the authors and pointed out the limitations of purely textual memory. This paper was produced using Pine Copilot's voice-directed *whisper coding* workflow [Pine AI, 2026], in which the authors specify, discuss, and review the work by voice while a coding agent (Claude Code with Claude Opus 4.8) carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

A Detailed results

This appendix collects the per-cell numbers behind Sections 5–6. Recall is top-1 with the argmax restricted to registered markers. Tables were measured on different face pools, splits, and draw protocols (single-draw sweeps versus the 20-draw paired protocol below), so raw-cosine columns differ slightly across tables (e.g. 0.767 vs. 0.776 at N=1000); comparisons are valid within a table, not across tables. Tables marked *single eval draw* (Tables 9 and 10) are directional comparisons on one shared draw; the paired multi-draw protocol below is the statistical reference.

The 12 PerceptMem domains. Table 4 enumerates the benchmark behind Figure 10: each domain is a (dataset, encoder) pair, evaluated at $N \in \{10, 20, 40\}$ with 30 draws each (90 tasks per domain, 1,080 total). The five conceptual sub-modalities of Table 2 each appear under at least one dataset–encoder pairing, several under two, plus the VLM's native vision tokens as the deliberately weak twelfth key space.

Paired evaluation protocol. A comparison between the memory and retrieval requires two controls: both methods score on *identical* registrations and queries, and each target identity occupies a randomly chosen bank slot. The second control matters because a fixed target slot lets the constant marker token in that slot accrue a baseline output-logit, which inflates recall independently of the query. We score both methods on the same twenty draws with the target slot randomised, and report mean$\pm$std. Table 5 gives the result, with random-bank

Table 4. The 12 PerceptMem domains: dataset, identity key encoder, cross-condition identity pool, and parametric recall@1 at N=20 (mean ±95% CI over 30 draws; chance 0.05). Tone identities are (speaker, emotional-state) pairs. The Qwen2.5-VL native-token row is included as a key-space control, not a recommended configuration (§6).

Domain	Key encoder	Pool	Recall@1, N=20
Face / AgeDB (cross-age)	ArcFace	500	0.869 ± 0.021
Face / LFW	ArcFace	1680	0.932 ± 0.018
Face / LFW	AntelopeV2	901	0.953 ± 0.012
Face / LFW+AgeDB combined	ArcFace	2180	0.921 ± 0.019
Speaker / LibriSpeech	ECAPA-TDNN	58	0.992 ± 0.004
Speaker / VoxCeleb (in-wild)	ECAPA-TDNN	40	0.991 ± 0.005
Acoustic scene / ESC-50	AST	50	0.792 ± 0.023
Painting style / WikiArt	CLIP (mid-layer)	128	0.319 ± 0.024
Painting style / WikiArt	DINOv2 [Oquab et al., 2024]	50	0.236 ± 0.023
Vocal tone / RAVDESS	wav2vec2 (emotion)	168	0.496 ± 0.021
Face / AgeDB (control)	Qwen2.5-VL native tokens	567	0.542 ± 0.029
Painting style / WikiArt	contrastive head	26	0.245 ± 0.018

face rows spanning N=10 to N=1000. The memory ties retrieval at every size and in every regime, since with a sharp read it computes the same argmax over encoder cosine — even on banks of the nineteen most-confusable look-alikes, where the cosine gaps are smallest (0.853 vs. 0.853, Δ=0.0pp). Recall is not monotone in N (0.749 at N=300 vs. 0.776 at N=1000): a draw at N=1000 spans nearly the whole 1090-identity eval pool, whereas smaller N are random subsets whose average difficulty differs from the pool's; the effect is identical in both columns, so it is a property of the encoder and the pool, not of the memory. The sharpness is load-bearing: an earlier trained variant whose learned temperature settled at inv-temp ≈ 20 trailed retrieval by eight points on the look-alike cell and by nine to eighteen points at $N \geq 300$, because a soft blend of marker values carries no per-identity signal beyond the cosine it dilutes. With the constants of Section 3 the blend is a faithful argmax in every cell we measure.

Table 5. Paired evaluation: the memory and retrieval scored on identical registrations and queries across 20 draws, with the target slot randomised. The memory ties retrieval everywhere — random banks through N=1000, style banks, and adversarial banks of the nineteen most-confusable look-alikes. Recall is the encoder's throughout.

Regime	Cell	Retrieval	AttMem (mean±std)	Δ
random	Face, N=10	0.948	0.948 ± 0.033	0.0pp
random	Face, N=100	0.792	0.792 ± 0.021	0.0pp
random	Face, N=300	0.749	0.749 ± 0.018	0.0pp
random	Face, N=1000	0.776	0.777 ± 0.008	0.0pp
random	Style, N=5	0.473	0.473 ± 0.104	0.0pp
random	Style, N=10	0.428	0.428 ± 0.076	0.0pp
adversarial	Face, K=19	0.853	0.853 ± 0.007	0.0pp

Universality grid. Table 6 runs the training-free read across five model architectures and all five modalities, scoring each cell against the encoder with the same paired protocol (20

draws). Every one of the twenty-five cells reproduces the encoder's recall to within one point (worst case 0.010, most cells ≤ 0.003). Tied-embedding models use the default read-strength gain; untied and hybrid models use a single larger value (256), set once per model and held fixed across modalities. The read therefore inherits the encoder's recall regardless of the host model's family, embedding scheme, scale, or even whether it is a transformer.

Table 6. Universality grid: maximum $|\Delta|$ between the training-free memory and the encoder over $N \in \{5, 10, 20\}$ (paired, 20 draws), for five architectures $\times$ five modalities. Every cell is within one point of the encoder. "gain" is the single per-model read-strength constant. Encoder recall@1 at N=10: Face 0.95, Speaker 0.99, Acoustic 0.86, Tone 0.52, Style 0.43.

Model	Emb.	gain	Face	Speaker	Acoustic	Tone	Style
Qwen2.5-7B	tied	64	0.001	0.000	0.002	0.003	0.001
Phi-3.5-mini	tied	64	0.001	0.000	0.000	0.002	0.002
DeepSeek-Llama-8B	untied	256	0.000	0.000	0.002	0.002	0.000
Mistral-7B	untied	256	0.000	0.000	0.003	0.010	0.000
Granite-4.0 (Mamba)	hybrid	256	0.003	0.000	0.003	0.001	0.000

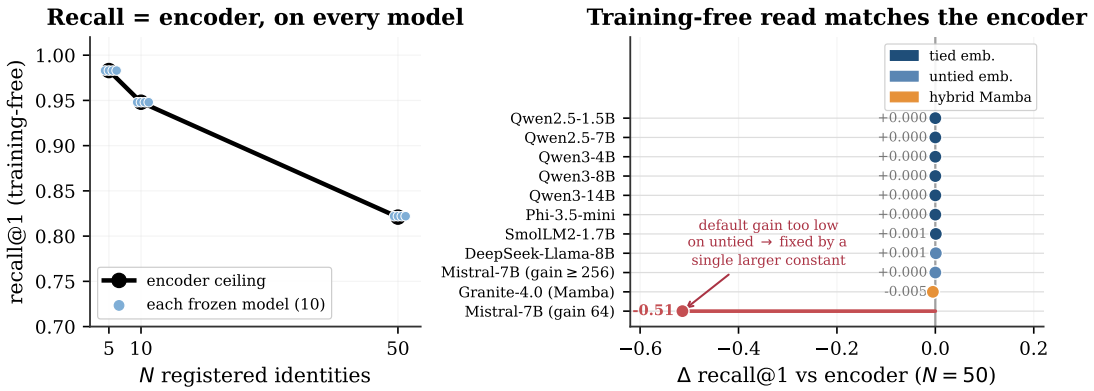


Figure 16. The training-free read reproduces the encoder on every frozen model, the supporting property behind the "store" step. **Left:** each model's training-free recall lands on the encoder ceiling (paired, 20 draws). **Right:** the gap from the encoder at N=50 is ≈ 0 across ten families (1.5B–14B, tied and untied embeddings, hybrid-Mamba); only an untied model at the default gain dips (red), fixed by a single larger read-strength constant. No gradient steps anywhere.

Read-strength robustness. The one per-model constant, the residual gain, is set by inspection rather than search because exact recall holds across a wide range (Table 7, face N=50, paired). A tied-embedding model is exact at every gain from 16 to 1024; an untied one (Mistral) is below the encoder until a threshold near 128–256 and then exact and stable through 1024. Any value in the upper range works for either, so there is no tuning to get wrong.

Capacity law across modalities. Table 8 runs the perceptual capacity probe (compress M registered identities into k prototype slots, recognise a cross-condition query, 5 seeds) on all five modalities. The slot-compression factor $\min(1, k/M)$ fits every modality once normalised by the one-slot-per-identity ceiling $C(M)$: the normalised error is ≤ 0.012 on the strong-encoder

Table 7. Read-strength robustness: training-free recall@1 (face, N=50, encoder ceiling 0.821) as the residual gain sweeps 16–1024. Tied models are exact throughout; untied models are exact above a threshold and stay exact. The constant tolerates a wide range.

Model	16	32	64	128	256	512	1024
Qwen2.5-3B (tied)	0.821	0.821	0.821	0.821	0.821	0.821	0.821
Mistral-7B (untied)	0.022	0.060	0.307	0.768	0.821	0.821	0.821

modalities and ≤ 0.13 on the weaker style and tone encoders. The ceiling $C(M)$ itself is the encoder's recall and varies accordingly, from near 1 on voice to 0.26–0.36 at M=32 on style and tone. Recognition is capacity-limited and graceful in every modality; only the ceiling moves.

Table 8. Perceptual capacity law per modality. $C(M)$ is the one-slot-per-identity ceiling (recall at k=M), i.e. the encoder's recall at M. "fit" is the mean $|\,\text{recall}/C(M) - \min(1, k/M)\,|$ over the k<M cells: the slot-compression factor is universal; the ceiling is the encoder's.

Modality	$C(2)$	$C(8)$	$C(32)$	**fit to** $\min(1, k/M)$
Face (ArcFace)	0.98	0.95	0.92	0.002
Voice (ECAPA)	1.00	1.00	0.99	0.001
Acoustic (AST)	0.95	0.87	0.76	0.012
Style (CLIP)	0.76	0.46	0.26	0.070
Tone (wav2vec)	0.94	0.67	0.36	0.125

Learned-metric baseline. A natural question is whether *any* re-encoding of the same features beats raw cosine, even if our read does not. We fit a similarity on the same encoder features and the same identities (the 50/50 split, seed 42) and score it with the identical protocol. Two closed-form metrics suffice: regularised LDA and within-class whitening. Table 9 shows LDA edging raw cosine at small memory and falling below it past a few hundred identities, while whitening never moves more than a point from raw cosine in either direction. So there is no meaningful discrimination left to extract from these features: the encoder's cosine is already the best ruler, which is why the memory's inheriting it exactly is the ceiling, not a shortfall. A contrastive linear head we also trained underperformed both closed-form metrics and is omitted as a weak instantiation.

Comparison with per-concept methods. The personalized-VLM line (MyVLM, Yo'LLaVA, Online-PVLM) registers a concept by *learning* something per concept (a classifier head, a soft token, a projection). Their code and checkpoints are unreleased, so we reimplement each method's core mechanism charitably on the cross-condition face task (ArcFace keys, same split). Table 10 reports accuracy and registration cost. Two findings. First, a per-concept linear classifier (MyVLM's mechanism) is indistinguishable from raw cosine at every memory size (within single-draw noise), because with one registration embedding per identity the trained head collapses to the embedding itself. It yields no accuracy gain over retrieval, and it costs per-identity gradient descent that grows to 4.5 s at a thousand identities, against our $\mathcal{O}(1)$ tensor append. Second, a learned projection trained with supervised contrastive loss (Online-PVLM's mechanism) *underperforms* raw cosine by 15 to 28 points on this task. We read this as a caution that a learned re-encoding is not without cost: tuned on identity-disjoint data,

Table 9. Learned-metric baseline, recall@1, same encoder, same split, all methods scored on the same eval draw. LDA edges raw cosine at small N and falls below it at scale; whitening stays within a point of raw cosine throughout. No re-encoding of the same features meaningfully beats the cosine. ATTMEM's recall equals raw cosine (paired through N=1000, Table 5), so the raw-cosine column doubles as its recall.

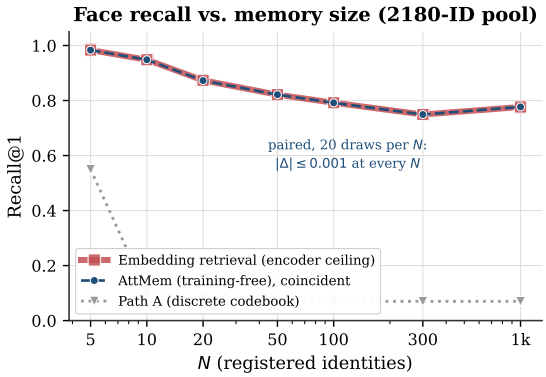
Modality	N	Raw cosine (= ATTMEM)	LDA	Whitening
Face (ArcFace)	5	0.933	**1.000**	0.933
Face (ArcFace)	10	0.933	**0.967**	0.933
Face (ArcFace)	300	0.734	0.667	**0.744**
Face (ArcFace)	1000	0.767	0.724	**0.776**
Style (CLIP-mid)	5	0.400	0.200	**0.600**
Style (CLIP-mid)	10	**0.400**	0.333	0.367

it can distort the very cross-condition geometry it was meant to sharpen. No method here, ours included, beats raw cosine on accuracy at scale; the parametric memory's case rests on cost and in-model behaviour, not recall.

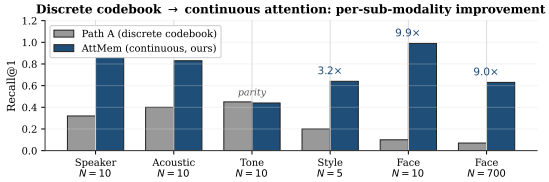
Table 10. Charitable reimplementations of per-concept methods on cross-condition face recall (ArcFace, recall@1, single eval draw). MyVLM's per-concept classifier ties raw cosine but its registration cost grows with the memory; Online-PVLM's learned projection underperforms raw cosine. ATTMEM's recall equals raw cosine (paired through N=1000, Table 5) at $\mathcal{O}(1)$ registration cost, so the raw-cosine column doubles as its recall.

N	Raw cosine (= ATTMEM)	MyVLM	Online-PVLM	MyVLM reg. cost
5	0.933	0.933	0.800	0.009 s
10	1.000	0.967	0.750	0.019 s
50	0.767	0.753	0.675	0.178 s
100	0.780	0.770	—	0.528 s
300	0.728	0.725	—	1.39 s
1000	0.776	0.778	—	4.52 s

Scaling and the codebook comparison. Figure 17a traces recall against memory size on the face pool under the paired protocol (20 draws per N). The training-free read is coincident with the encoder ceiling at every size — 0.948 vs. 0.948 at N=10, 0.777 ± 0.008 vs. 0.776 ± 0.008 at N=1000, $|\Delta| \leq 0.001$ throughout (Table 5) — so the memory declines exactly as the encoder's cosine does, gracefully, from near-perfect at ten identities to 0.78 at a thousand. The coincidence requires the sharp read of Section 3: with a soft read (learned inv-temp ≈ 20) the same sweep trails the cosine by nine to eighteen points at $N \geq 300$, as recorded in the paired-protocol paragraph above. The discrete-codebook predecessor (PATH A), by contrast, flatlines at $\sim$0.07 regardless of codebook size or budget. Figure 17b makes the same point across all five sub-modalities: replacing a discrete codebook with continuous attention lifts recall 2–10$\times$, because a categorical bottleneck discards the signal the encoder preserved. This is the one place a design choice in the read matters; everything else inherits the encoder.



(a) scaling on the face pool



(b) codebook vs. continuous attention

Figure 17. Left: under the paired protocol (20 draws per N) the training-free read is coincident with the encoder ceiling from N=5 to N=1000 ($|\Delta| \leq 0.001$; Table 5), both declining gracefully as the pool grows, while the codebook predecessor never gets off the floor. Right: continuous attention beats the discrete codebook 2–10× on every sub-modality.

Vision-language model, key/value orthogonality. Table 11 gives the numbers behind the structural rule of Section 6. Each memory row is compared to retrieval *in its own key space*, under the training-free sharp read. An external ArcFace key (orthogonal to the model's hidden space) reproduces the encoder's recall exactly at every size — 0.948/0.792/0.776 at N=10/100/1000, the same values as on the text-LM hosts, as the universality result predicts. The VLM's native vision tokens (already in hidden space) also tie their own retrieval — the read is faithful to whatever key it is given — but that ceiling sits far below ArcFace's throughout, so they are the wrong key regardless of method: the empirical basis for reserving the VLM for localization and an external encoder for identity.

Table 11. Parametric memory inside Qwen2.5-VL-3B: same frozen model, only the key encoder changes; training-free sharp read throughout. ArcFace rows: paired protocol, 20 draws. Native-token rows: single draw on AgeDB with this harness's own key pooling, so they differ from the AgeDB sweep of Section 6 (0.64/0.34); both agree that native tokens sit far below ArcFace. The read ties retrieval in both key spaces; only the key's own ceiling differs. An earlier trained variant of the ArcFace row (12k steps, learned soft temperature) trailed retrieval at scale (0.494 at N=1000) — the same under-sharpening artifact recorded in the paired-protocol paragraph above.

Configuration	N=10	N=100	N=1000
Retrieval over ArcFace embeddings (encoder ceiling)	0.948	0.792	0.776
AttMem + external ArcFace keys (orthogonal to hidden space)	0.948	0.792	0.776
Retrieval over native vision tokens	0.50	0.43	—
AttMem + native vision tokens (co-located with hidden space)	0.50	0.42	—

Realistic-scene stress test. The composites of Section 5.2 place each face in its own well-separated cell, which is where grounding is near-lossless. Table 12 stresses the grounding step instead: faces are pasted onto a dimmed photographic background at jittered positions and varying scales (0.7–1.05×), in a one-row or two-dimensional grid layout, and the referent is named by reading order (Qwen2.5-VL-7B, M=40, K=3, 3 seeds per layout). Grounding

accuracy drops from 1.00 on clean composites to 0.81 (row) and 0.63 (grid), and end-to-end recall falls well below its oracle because the face detector also degrades on small, rescaled faces (hit-rate 0.09–0.43, forcing un-aligned fallback crops). This is the basis for limitation (iii): in cluttered photographic scenes the grounder and detector, not the encoder, are the practical bottleneck.

Table 12. Realistic-scene stress test: faces composited onto photographic backgrounds with positional jitter and scale variation (Qwen2.5-VL-7B, M=40, K=3; mean over 3 seeds). The correct-region oracle stays high, while VLM grounding and detector alignment degrade — the failure mode is localization, not identity.

Layout	Oracle region	Grounded (ours)	Whole-scene	Grounding acc.
Row (jittered)	0.83	0.42	0.03	0.81
Grid (jittered)	0.83	0.27	0.03	0.63

B Mechanics of a single recall

It helps to watch one recall end to end. Suppose the agent has met ten people and wants to register an eleventh from a single photo. Registration is one row appended to the face bank (Listing 1). The encoder turns the photo into a 512-dimensional ArcFace embedding, the *key*. The model's own embedding for a freshly assigned marker token becomes the *value*, a vector of the model's hidden width (2048 for Qwen2.5-3B). There is no gradient step and no fine-tuning. The row is the memory.

```
# Register identity #11 from a single photo -- no training:
k = l2_normalize(arcface(photo))        # key:   R^512  (encoder space)
m = assign_marker_token()               # e.g. "<id_11>"
v = model.input_embedding[m]            # value: R^2048 (model's own space)
bank.K = torch.cat([bank.K, k[None]])   # O(1) append
bank.V = torch.cat([bank.V, v[None]])
```

Listing 1. One registered identity is one row. The key is the encoder's view of the perception; the value is the model's own vector for the marker token that names it. Adding the row is a single tensor append.

At recall, a new photo of one of the eleven arrives under different lighting and a year older. Its ArcFace embedding becomes the query $\boldsymbol{q}$. Just before its output head, the model scores $\boldsymbol{q}$ against all eleven keys, softmaxes into attention weights $\boldsymbol{w}$, and pulls back the weighted blend of values $\boldsymbol{r} = \boldsymbol{w}^\top \boldsymbol{V}$. Because each value is the model's own vector for a marker token, adding $g\, W_o \boldsymbol{r}$ to the hidden state lands as a clean boost on the right marker's logit. The model's next token *is* the remembered identity. The whole step is a few matrix multiplies dwarfed by the model's forward pass.

The 0.98-vs-0.46 diagonal gap of Section 5 (Figure 12) is measured on exactly this probe: a held-out ten-identity bank, comparing the sharp attention weights against the raw encoder cosine they are computed from. The attention concentrates the cosine onto its top match, which is why the read recovers the encoder's nearest neighbour as a single decisive marker rather than a diffuse blend.

C Binding exact facts in a latent memory

Our original exact-fact experiment (Figure 15, right) found that a latent of k=16 soft tokens holds one six-character code (0.87) but collapses at two (0.06). Those runs were trained for 3,000 steps on one seed. This appendix tests whether the collapse is a limit of the latent or of the training budget. In short, it is mostly the budget: a shared latent can bind two codes and, less reliably, three. Per-item slots help further but do not guarantee binding. Every longer-trained result below has two seeds (42 and 43), and they often disagree, so we report both. The code-length grid at the end has one seed. The code, per-run results and a map from each number below to its source are in `results/binding_followup/`.

Setup. Everything except the training schedule is the original experiment, with the same model class imported unchanged. A frozen Qwen2.5-1.5B-Instruct reads a document of M lines `Name: CODE`, where each code has six characters from a 32-symbol alphabet. A write head of 16 learned queries cross-attends (8 heads) to the model's layer-14 hidden states and produces the k=16 latent tokens. A LoRA reader (rank 32 on the attention projections) answers the prompt `Name:` from the latent alone. Each training batch draws a document size M' uniformly from 1..M. We use AdamW at learning rate 2×10^{-4} with batch 8. We compare three arms. The *original protocol* runs 3,000 steps with a cosine schedule to zero, on seeds 42, 43 and 44. *Trained longer* keeps the learning rate constant after warm-up and stops once recall on M-code documents has not improved by 0.01 for 4,000 steps (never before 8,000 steps; all but the first seed-42 runs also require the training loss to have stalled), then anneals linearly over 1,000 steps. Multi-code runs are capped at 40,000 steps; one-code runs stop on the training loss, with a cap of 15,000. The *per-line mask* arm is described below. Recall is teacher-forced exact match on 512 fresh documents: a code counts only if every one of its tokens is the model's top prediction given the correct preceding tokens. All rank-16 runs were capped at 15,000 steps and all reached the cap; the seed-42 ones came from an earlier revision of the script that stopped on the training loss.

Table 13. Exact-match recall of M codes from k=16 latent tokens. Each cell is a model trained with up to M codes and tested on M-code documents. Longer-trained cells give seed 42 / seed 43. Training steps (s42 / s43): rank 32, 15k for one code, 26k / 26.5k for two, 24.5k / 19.5k for three; rank 16, 15k throughout (cap); per-line mask, 40k / 24k for two and 40k / 40k for three.

Arm	M=1	M=2	M=3
Original runs (3k steps, one seed)	0.869	0.063	–
Rerun, 3k (seeds 42/43/44)	0.648/0.395/0.951	0.000/0.037/0.000	0.027/0.000/0.014
Trained longer, rank 32	0.982 / 0.992	0.555 / 0.400	0.135 / 0.230
Trained longer, rank 16	0.996 / 0.992	0.715 / 0.711	0.311 / 0.410
Per-line mask, rank 32	–	0.893 / 0.998	0.992 / 0.639

Training length. At 3,000 steps the rerun reproduces the collapse on all three seeds (Table 13). One-code recall at that budget ranges from 0.40 to 0.95 across seeds, and three of the nine 3,000-step models had not yet left the initial loss plateau. Both are signs that 3,000 steps falls in the middle of the learning curve rather than at its end. Trained longer, every shared two-code model recalls one code at 0.9 or better within 4,000 steps. Two-code recall then climbs slowly

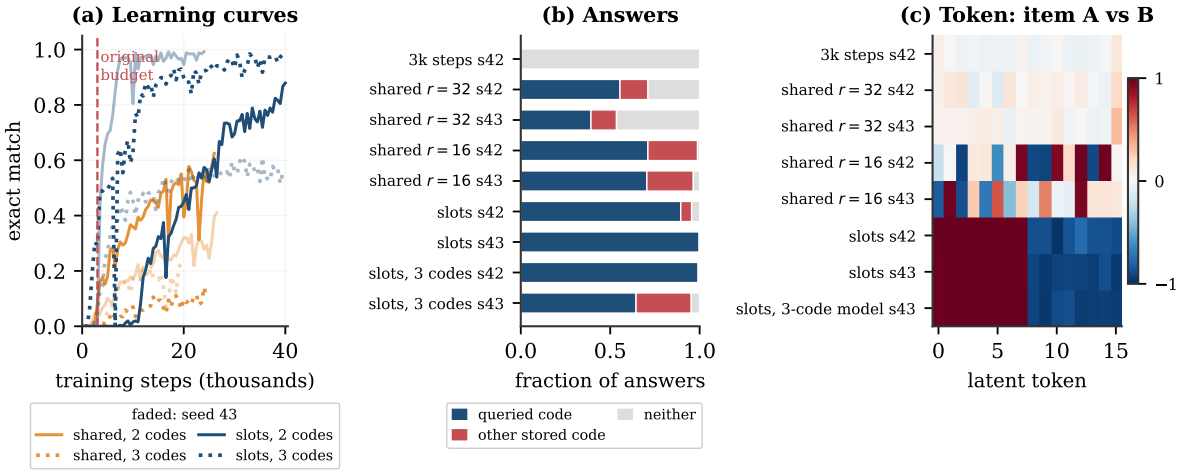


Figure 18. Binding in the exact-fact latent, seeds 42 and 43. **(a)** Recall during training (256 held-out documents, evaluated every 500 steps); solid lines are seed 42, faded lines seed 43. The shared latent learns two codes slowly and falls off at three. With per-line slots, three codes are learned within a few thousand steps on seed 42 but stall near 0.5 on seed 43. The dashed line marks the original 3,000-step budget. **(b)** Greedy answers, on two-code documents unless marked "3 codes". Trained models often answer with another stored code intact; at rank 16, and for the seed-43 three-code slotted model, nearly every error is such a swap. **(c)** For each of the 16 latent tokens of a two-item document AB: its cosine with the same token of the one-item latent for A, minus that for B. Values near zero mean the token mixes both items; ± 1 means it copies one. Among the six-character shared latents shown, only the rank-16 models develop item-specific tokens (a rank-32 model with eight-character codes does too; Table 15); the per-line mask has them by construction, including in the seed-43 three-code model that still mis-binds.

and noisily (Figure 18a). At rank 32 it first passes 0.5 at 16,500 steps on seed 42 in the in-training evaluation, and on seed 43 it never does (best 0.41 in 26,500 steps). The final scores are 0.555 and 0.400 on fresh documents. The seed-42 model stopped earlier by a loss-based rule (18,500 steps) gives 0.541, so these are lower bounds on what longer training reaches. At rank 16, two-code recall passes 0.5 at 7,000 and 8,000 steps.

Storage versus binding. Two measurements separate holding the codes from pairing them with names (Table 14). A *slot probe* is a linear map trained to read the code on line i from the latent, scored by per-character accuracy (chance 0.03; a probe on an untrained write head reaches 0.22–0.25). It recovers both codes of a trained two-code latent at 0.79–0.90, against 0.92–0.94 for a one-code latent. Both codes are therefore stored. Many of the model's own errors are swaps. At rank 32, 14–16% of answers are the other stored code, a quarter to a third of all errors. At rank 16, 96–99% of answers are one of the two stored codes, intact, and 26–28% are the wrong one, so 88–96% of errors are binding errors: the right content returned for the wrong name. A name-conditioned probe, which is given the latent and the queried name, fails even where the language model reads perfectly. It is too weak to say anything about binding, so we rely on the model's own answers.

How the latent arranges two items. At 3,000 steps the two-item latent is close to a symmetric sum of the two one-item latents. A per-document least-squares fit $L_{AB} \approx \alpha L_A + \beta L_B$ explains R^2=0.76–0.88 of it (on seed 42, $\alpha \approx \beta \approx 0.42$), and swapping the line order barely changes it

Table 14. Binding analysis on two-code documents. Answers use greedy decoding on 512 documents and give the fraction that are the queried code, the other stored code, or either one. "Slot probe" is the per-character accuracy of a linear probe reading the code on line 1 / line 2 from the latent. $\cos(L_{AB}, L_{BA})$ compares the latents of the same two lines in either order; high values mean the latent barely depends on which line came first.

Model	seed	queried	other code	any stored	slot probe	$\cos(L_{AB}, L_{BA})$
3k steps, rank 32	42	0.000	0.000	0.000	0.34 / 0.45	0.93
	43	0.041	0.029	0.070	0.73 / 0.79	0.79
Trained longer, rank 32	42	0.555	0.156	0.711	0.79 / 0.83	0.81
	43	0.393	0.143	0.535	0.85 / 0.87	0.83
Trained longer, rank 16	42	0.711	0.277	0.988	0.90 / 0.90	0.32
	43	0.705	0.260	0.965	0.86 / 0.88	0.49
Per-line mask, rank 32	42	0.895	0.061	0.955	0.81 / 0.80	−0.03
	43	0.998	0.000	0.998	0.89 / 0.92	0.00

(cos 0.79–0.93). Both rank-16 two-code models broke this symmetry on their own. On seed 42, four tokens copy item A and four copy item B (cosine $\geq$ 0.92 with one item and $\leq$ 0.06 with the other). On seed 43, two tokens copy A and three copy B (cosine $\geq$ 0.82 with one item, $\leq$ 0.10 with the other). Their latents therefore depend on order (cos 0.32 and 0.49), and their errors are clean swaps. With six-character codes, both rank-32 two-code models and both rank-32 three-code models kept all 16 tokens as roughly even mixtures of the items (cos 0.81–0.88), and they bind worst (Figure 18c). The rank-16 three-code model on seed 43 is partly separated (one token matches A at 0.95 and B at 0.12) and is the best shared three-code model with six-character codes. Across these runs, on both seeds, a shared latent binds about as well as it separates items into their own tokens. The split is not specific to rank 16: a rank-32 model with eight-character codes made it too (Code length, below). Separation is not sufficient, though, as the per-line mask shows below.

Three codes. With three codes the shared rank-32 latent reaches only 0.135 (seed 42, 24,500 steps) and 0.230 (seed 43, 19,500 steps). On seed 42, two other stopping rules, one based on loss and one on accuracy alone, give 0.129 and 0.117, so the stopping rule is not what limits it. Rank 16 reaches 0.311 and 0.410 at its 15,000-step cap. Under the original protocol the three-code models scored 0.00–0.03.

Per-line mask. This variant changes only which document tokens the write head may attend to. The 16 latent queries are split into M contiguous groups (8/8 for two lines; 6/5/5 for three). A hard boolean attention mask lets group j attend only to the tokens of line j (`Name: CODE` and its newline) and to nothing else. Parameters, reader, loss and data are unchanged. The writer must therefore know the line boundaries when it writes the memory, which the shared latent does not need. Each group of latent tokens is then a function of exactly one item, giving a hard slot-per-item memory at the same k=16.

On three of four runs the mask binds (near-)perfectly. With two codes it reaches 0.893 on seed 42 (at the 40,000-step cap and still rising) and 0.998 on seed 43 (24,000 steps), with at most 6% wrong-code answers. With three codes on seed 42 it passes 0.5 within 4,500 steps and reaches 0.992 with no wrong-code answers. The fourth run, three codes on seed 43, does not. Its recall passes 0.5 only at 16,000 steps, then stays between 0.45 and 0.59 until the 40,000-step

cap, and ends at 0.639. Its latent is cleanly slotted: on two-item documents, the eight tokens of A's group match A's one-item latent at cosine 1.00, the eight of B's group match B's at 0.88–0.96, and the latent is order-independent (cos -0.01). Yet 31% of its answers are another stored code, intact (87% of its errors), and 95% are some stored code. Storage is separated; what failed is the reader's mapping from the queried name to the right slot. Hard slots therefore remove interference between items in the latent but do not by themselves guarantee binding.

The initial plateau also varies with the seed: with two codes, recall stays at zero for about 7,000 steps on seed 42 but leaves zero within 4,000 steps on seed 43. An accuracy-only stopping rule halted one seed-42 run inside that plateau at 9,000 steps with 0.01, which is why later runs also require the loss to stall. Slots do not remove the need for training: a rank-16 masked two-code model trained for 3,000 steps scores 0.000.

Rank. Rank 16 beat rank 32 on both seeds, with two codes (0.715/0.711 vs. 0.555/0.400) and with three (0.311/0.410 vs. 0.135/0.230), in fewer steps (15,000 vs. 19,500–26,500). On both seeds its two-code latent split into item-specific tokens and rank 32's did not. The seed-43 rank-16 runs used the current script, so the seed-42 difference in script revision and stopping rule does not explain the gap. A lower rank may make the split more likely, perhaps by putting more pressure on the write head to separate items, but we have not tested that, nor intermediate or higher ranks. The split itself is not tied to rank 16: the rank-32 two-code model with eight-character codes formed item-specific tokens on its own and bound better than any other rank-32 shared model (Table 15). In our runs the split, not the rank, is what goes with good binding; with two seeds at rank 16 and one split model at rank 32, we read this as an observation, not an established effect.

Code length. Table 15 repeats the shared-latent, rank-32 arm with codes of two, four and eight characters, on seed 42 only, with the same stopping rules; the six-character row is the seed-42 run above. One code is recalled at 0.98 or better at every length. Three codes degrade steadily with length, from 0.604 at two characters to 0.088 at eight. At six and eight characters this is partly a storage failure: a linear slot probe reads lines 2 and 3 of a three-code latent at only 0.46–0.58 per-character accuracy, and only 14–21% of answers are any stored code. At two characters nearly every answer is a stored code (92%), and the errors are swaps (33% of answers are the wrong stored code).

Two codes do not follow length: 0.721, 0.635 and 0.555 for two, four and six characters, then 0.752 for eight. The eight-character model trained longest (38,000 steps) and, at rank 32, split its latent into item-specific tokens without being told to. Five tokens copy item A (cosine ≥ 0.99 with A, ≤ 0.04 with B) and five copy item B (≥ 0.89 with B, ≤ 0.04 with A), and $\cos(L_{AB}, L_{BA})$ is 0.19, against 0.81–0.85 at the other lengths. Its errors are then almost all swaps: 99% of answers are a stored code and 24% are the wrong one. Item-specific tokens therefore form spontaneously in some runs, at rank 32 as well as rank 16.

Across all our runs the pattern is consistent. The three shared latents that split into item-specific tokens (rank 16 on both seeds, and this eight-character model) put 96–99% of answers on a stored code, with 24–28% of answers the wrong one. Latents that stay mixed both corrupt codes and swap them. Separating items fixes storage; the remaining error is binding the queried name to its slot, the same failure that limits the seed-43 three-code mask model. All cells in Table 15 come from one seed; given the seed spread above, any of them could move by

about 0.15.

Table 15. Code length, shared latent trained past the plateau, LoRA rank 32, seed 42. Recall is teacher-forced exact match on 512 fresh documents for a model trained with up to M codes. "Other / any" are greedy answers that are the wrong stored code or any stored code. $\cos(L_{AB}, L_{BA})$ is for the two-code model; a low value means its latent has item-specific tokens. The six-character row repeats the seed-42 runs of Table 13.

chars	M=1	M=2	M=3	M=2 **other / any**	M=3 **other / any**	$\cos(L_{AB}, L_{BA})$	**steps (**M=2 **/** 3**)**
2	1.000	0.721	0.604	0.19 / 0.91	0.33 / 0.92	0.85	19.5k / 22k
4	1.000	0.635	0.482	0.18 / 0.81	0.26 / 0.74	0.83	17k / 25.5k
6	0.982	0.555	0.135	0.16 / 0.71	0.08 / 0.21	0.81	26k / 24.5k
8	0.986	0.752	0.088	0.24 / 0.99	0.06 / 0.14	0.19	38k / 28.5k

Scope. Codes have two to eight characters, with at most three items, one per line. The code-length grid has one seed. We have not tested more than three items, free-form text, or a k sweep beyond the original 3,000-step budget.

D A discrete-codebook alternative

Before continuous attention we spent sixteen development cycles on PATH A: a discrete codebook that snaps each perception to one of K learned codes and remembers the code. It is the natural "compress the perception into a symbol" design, and it is a learned cousin of captioning, which is why its failure is instructive rather than embarrassing.

No setting broke it. Across codebook sizes $K \in \{128, 256, 512, 1024\}$, even after *100,000* steps of continual pretraining on the expanded face pool, top-1 recall at 300 identities stays pinned between 0.057 and 0.070 while embedding retrieval over the same encoder reaches 0.73. That is a roughly 10× gap that does not move with K (Table 16). Swapping ArcFace R50 for AntelopeV2 R100 trained on 360K identities did not help either.

Table 16. PATH A after 100K-step continual pretraining on the 2180-identity face pool, at 300 registered identities. Recall is pinned near 0.07 regardless of codebook size, even when the gate routes to the correct code about half the time.

Codebook size K	128	256	512	1024
PATH A recall@1	0.057	0.064	0.070	0.070
gate routes to correct code	0.43	0.51	0.42	0.54
embedding retrieval (same encoder)		0.73		

The reason is a vice the codebook cannot escape, and the diagnostics name it exactly. Two pressures pull in opposite directions as K grows. A *small* codebook packs many people into each cell: at K=16, distinct identities collide in the same code 8.7% of the time, and once two people share a code they are indistinguishable forever after. A *large* codebook fixes that, with inter-identity collisions falling to 2.4% at K=128, but it shatters each identity across cells. The rate at which two cross-condition photos of the *same* person land in the same code drops from 0.33 to 0.20, so the query no longer routes to where the registration was stored. Net recall is

squeezed from both sides and never escapes ∼0.07. Even when the gate does route a query to the right code (about half the time, Table 16), every other identity sharing that cell is an equally good answer, so the final pick is near-random.

This is the same information loss as captioning, learned instead of written: any categorical bottleneck, whether a word or a code, throws away the continuous signal the encoder worked to preserve, and no amount of compute downstream can recover it. Continuous attention over the raw embedding never quantizes, and never pays this tax. PATH A is in the paper because its failure is the cleanest possible argument for the design that replaced it.

References

Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

Yuval Alaluf, Elad Richardson, Sergey Tulyakov, Kfir Aberman, and Daniel Cohen-Or. MyVLM: Personalizing VLMs for user-specific queries. In *European Conference on Computer Vision (ECCV)*, 2024.

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, et al. Flamingo: A visual language model for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Ruichuan An, Sihan Yang, Renrui Zhang, Ming Lu, Tianyi Jiang, Kai Zeng, Yulin Luo, Jiajun Cao, Hao Liang, Ying Chen, Qi She, Shanghang Zhang, and Wentao Zhang. MC-LLaVA: Multi-concept personalized vision-language model. *arXiv preprint arXiv:2411.11706*, 2024.

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Huiyu Bai, Runze Wang, Zhuoyun Du, Yiyang Zhao, Fengji Zhang, Haoyu Chen, Xiaoyong Zhu, Bo Zheng, and Xuejiao Zhao. Online-PVLM: Advancing personalized VLMs with online concept learning. *arXiv preprint arXiv:2511.20056*, 2025a.

Shuai Bai, Keqin Chen, Xuejing Liu, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025b.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, et al. SmolLM2: When smol goes big — data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022.

Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, et al. The AMI meeting corpus: A pre-announcement. In *International Workshop on Machine Learning for Multimodal Interaction (MLMI)*, 2005.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Joon Son Chung, Jaesung Huh, Arsha Nagrani, Triantafyllos Afouras, and Andrew Zisserman. Spot the conversation: Speaker diarisation in the wild. In *INTERSPEECH*, 2020.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Jiankang Deng, Jia Guo, Evangelos Ververas, Irene Kotsia, and Stefanos Zafeiriou. RetinaFace: Single-shot multi-level face localisation in the wild. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *INTERSPEECH*, 2020.

Yuan Gong, Yu-An Chung, and James Glass. AST: Audio spectrogram transformer. In *INTERSPEECH*, 2021.

Haoran Hao, Jiaming Han, Changsheng Li, Yu-Feng Li, and Xiangyu Yue. RAP: Retrieval-augmented personalization for multimodal large language models. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Rongpei Hong, Jian Lang, Ting Zhong, Yong Wang, and Fan Zhou. TAMEing long contexts in personalization: Towards training-free and state-aware MLLM personalized assistant. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2026. arXiv:2512.21616.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Gary B Huang, Marwan Mattar, Tamara Berg, and Eric Learned-Miller. Labeled faces in the wild: A database for studying face recognition in unconstrained environments. In *Workshop on Faces in Real-Life Images*, 2008.

IBM Granite Team. Granite 4.0: Hyper-efficient, high-performance hybrid models for enterprise. IBM announcement, 2025. URL https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.

Letta Team. Letta: Stateful LLM agents with long-term memory. Software framework, 2024. URL https://github.com/letta-ai/letta.

Bojie Li. Models take notes at prefill: KV cache can be editable and composable. *arXiv preprint arXiv:2606.17107*, 2026a.

Bojie Li. User as code: Executable memory for personalized agents. *arXiv preprint arXiv:2606.16707*, 2026b.

Bojie Li. User as engram: Internalizing per-user memory as local parametric edits. *arXiv preprint arXiv:2606.19172*, 2026c.

Bojie Li and Noah Shi. The latent bridge: A continuous slow–fast channel for real-time game agents. *arXiv preprint arXiv:2606.24470*, 2026.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, 2023.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Steven R Livingstone and Frank A Russo. The Ryerson audio-visual database of emotional speech and song (RAVDESS). *PLoS ONE*, 13(5):e0196391, 2018.

Lin Long, Yichen He, Wentao Ye, Yiyuan Pan, Yuan Lin, Hang Li, Junbo Zhao, and Wei Li. Seeing, listening, remembering, and reasoning: A multimodal agent with long-term memory. *arXiv preprint arXiv:2508.09736*, 2025.

Stylianos Moschoglou, Athanasios Papaioannou, Christos Sagonas, Jiankang Deng, Irene Kotsia, and Stefanos Zafeiriou. AgeDB: The first manually collected, in-the-wild age database. In *CVPR Workshops*, 2017.

Arsha Nagrani, Joon Son Chung, and Andrew Zisserman. VoxCeleb: A large-scale speaker identification dataset. In *INTERSPEECH*, 2017.

Thao Nguyen, Haotian Liu, Yuheng Li, Mu Cai, Utkarsh Ojha, and Yong Jae Lee. Yo'LLaVA: Your personalized language and vision assistant. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. LibriSpeech: An ASR corpus based on public domain audio books. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015.

Tae Jin Park, Naoyuki Kanda, Dimitrios Dimitriadis, Kyu J Han, Shinji Watanabe, and Shrikanth Narayanan. A review of speaker diarization: Recent advances with deep learning. *Computer Speech & Language*, 72, 2022.

Karol J Piczak. ESC: Dataset for environmental sound classification. In *ACM Multimedia*, 2015.

Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2026. URL https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice. Accessed 2026-06-28.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MemoryLLM: Towards self-updatable large language models. In *International Conference on Machine Learning (ICML)*, 2024.

WikiArt. WikiArt: Visual art encyclopedia. https://www.wikiart.org, 2010.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025.

Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.

Jin Xu, Zhifang Guo, Jinzheng He, et al. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025.

An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.